Books

1) The Complete Reference, Java 2

- Herbert Schild

-TMH

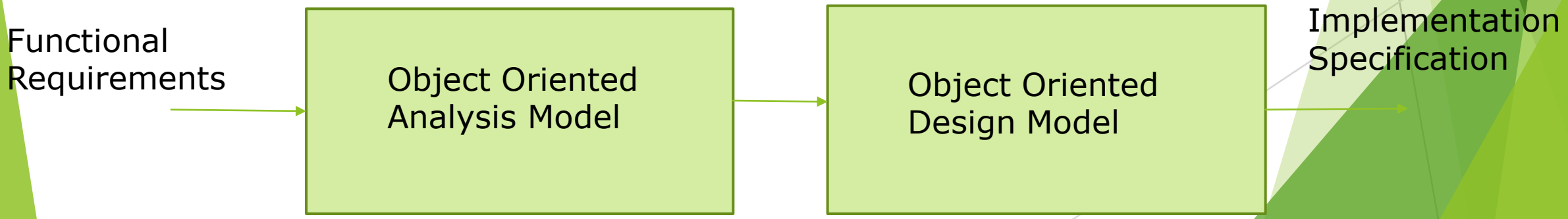
2) Object Oriented Modeling and Design with UML

- Michael Blaha and James Rambaugh

- Pearson

Introduction to OOAD

- Object oriented analysis and design is a software engineering approach which models the system as interacting objects.
- Each object represents a system entity which plays a vital role in building of that system.
- Object Oriented Analysis (OOA) focuses on analysis of functional requirements for the system.
- Object Oriented Design (OOD) takes analysis model as input and produces implementation specification.



Modeling

- There are three important type of UML models:
    1. Class Model
    2. State Model
    3. Interaction Model

Modeling

1) Class model

- The class model describes the structure of the object, the relationship of one object with other objects, attributes and operations of the object.
- The state and interaction models use the context of the class model.
- The goal of the class model is to capture the concepts from the real worlds that are useful for your application
- The class model is represented by the class diagrams. The classes in this diagrams define the attributes and operations of each object.

Modeling

2) State model

- The state model is concerned with the time and sequencing of the operations of the object.
- On occurrence of the events the object change their state.
- The state Model captures the control aspect of the system.
- The state model is represented by State diagram. Each state diagram represents state and the event sequences.

Modeling

3) Interaction Model

- Interaction model describes how one object collaborates with other in order to achieve behaviour of the system
- The overall behaviour of the system can be represented with the help of state and interaction model.
- The interaction model includes use case diagrams, sequence diagrams and Activity diagrams.
- The use case diagrams show how to outsider actor interacts with the system to achieve functionality.
- The sequence Diagram represents the objects that interact and the time sequence of their interaction
- The activity Diagram represents flow of control among objects.

CLASS MODELING

Introduction to class modeling

- A class model captures the static structure of a system by characterizing the objects in the system, the relationships between the objects and the attributes and operations for each class of objects.
- Class model provides a graphical representation of a system and are used for communicating with customers.

Class and Object Concept

Classes

- Class is a group of objects having same attributes and operations, relationships and semantics.
- The classes appear as common nouns or noun phrases.
- Objects in a class share a common semantic purpose. For Example both the dog and cat have the properties like tail and legs and they belong to same class Animal.
- Grouping the objects into corresponding classes make the design Abstract.

Objects

- The main purpose of class model is to describe objects.
- Object is an instance of a class.
- The objects can be conceptual entities, real world entities, or important things from implementation point of view.
- The objects are normally nouns. The choice of objects is done by judgements.
- For Example: If a student is a class then Anuja, pooja and Kajal are the objects of the class students. Each student has its own name, roll no, and address.

Class Diagrams

- The class Model is represented by two types of diagrams:-

1) Class Diagrams

- Class diagrams provide a graphic notation for modeling classes and their relationships thereby describing possible objects.
- Useful for abstract modeling and designing actual programs.
- They are concise ,easy to understand

2) Object Diagrams

- Object diagrams shows individual objects and their relationship .
- A class diagram corresponds to an infinite set of object diagrams.

Continue….

#Notation used for class & object diagram:

→For class diagram:-

Person

→For object diagram:-

Smith:Person **Mary:Person**

Values and Attributes

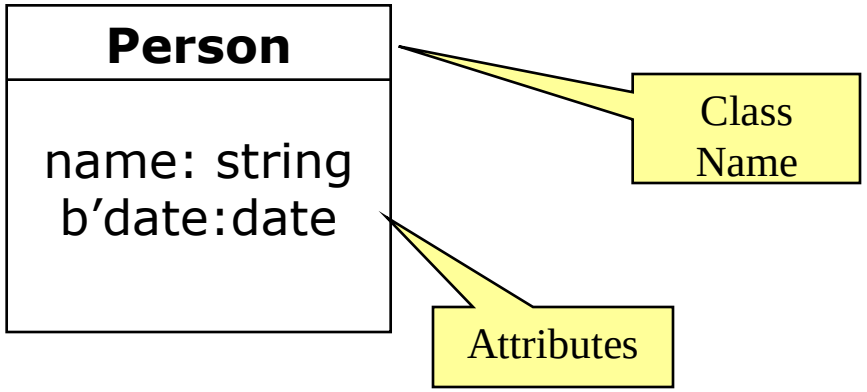
- A value is a piece of data.
- An attribute is a named property of a class that describes a value held by each object of the class.
- E.g. Name,birthdate and weight are attributes of Person class.

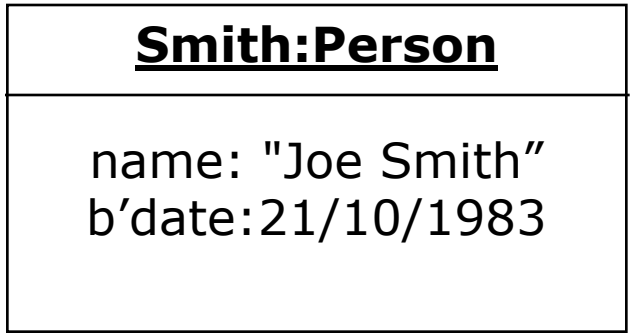
  - Color,modelyear and weight are att. Of Car class.
- Each attribute name is unique within a class.
- So Person and Car class have attribute called weight.

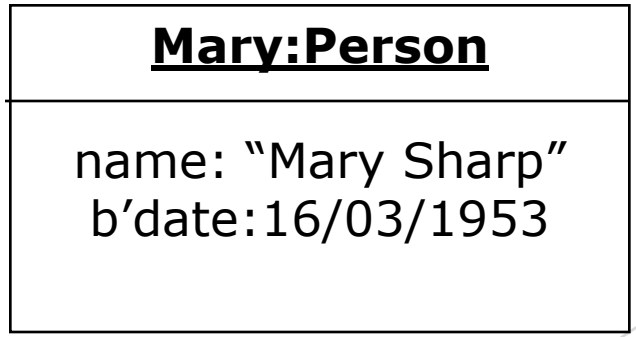
Values and Attributes continue….

 # Example:-

→List att. In the second compartment of the class box.







Operations and Methods

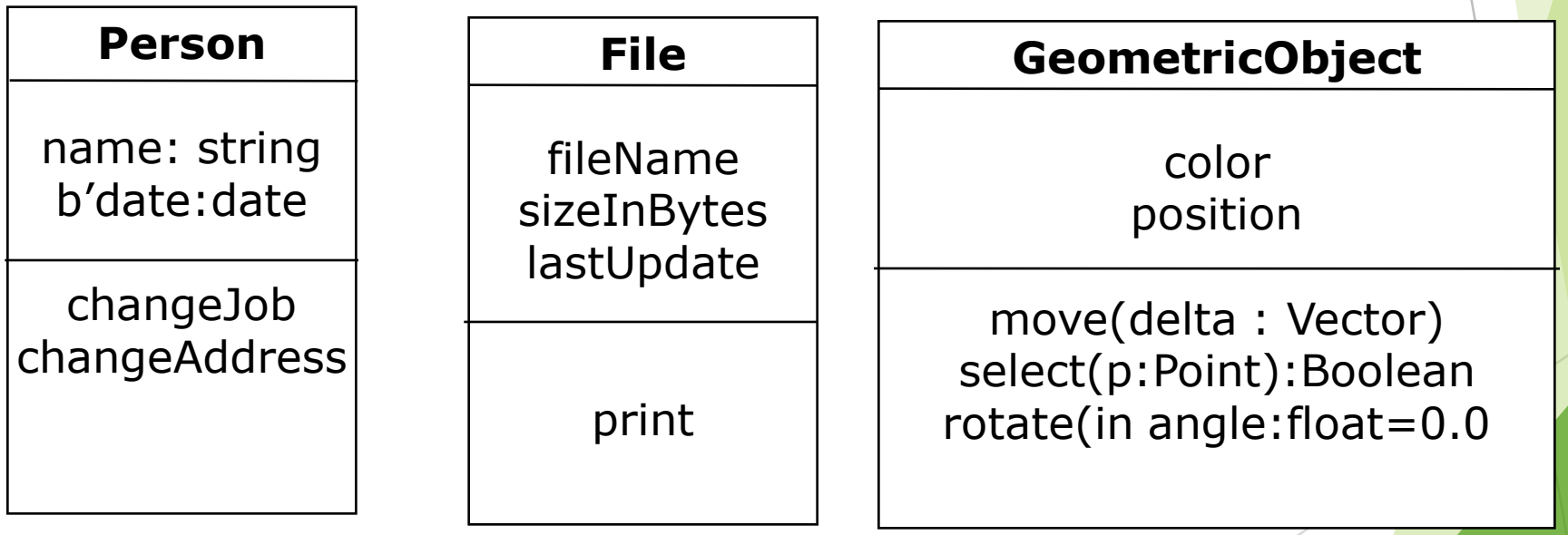
- Objects have procedures or functions which are called as **operations**.
- All the objects in the same class share the common set of operations.
- For Example- The class Shape can have various objects such as rectangle, triangle or square having **common operations such as move, draw, print.**
- **A method is the implementation of an operation for a class.**
- When an operation has methods on several classes it is important that the methods all have the same Signature.
- Signature means the number and types of arguments and type of return value.
- e.g. print should not have fileName as an argument for one method filePointer for another.

Operations and Methods continue…

→**Feature** is a generic word for either an attribute or operation.

→UML notation is to list operations in the third compartment of the class box.

#**Examples**:-



Summary of Notation for classes

→The attribute and operation compartments are optional.

→A missing attribute / operation compartment means that attributes / operations are unspecified.

→An empty compartment means attributes/operations are specified and that are none.

ClassName
attName1:dataType1=defaultValue1 attName1:dataType1=defaultValue1 . . .
operationName1(argList1) : resultType1 operationName2(argList2) : resultType2 . . .

Link and Association

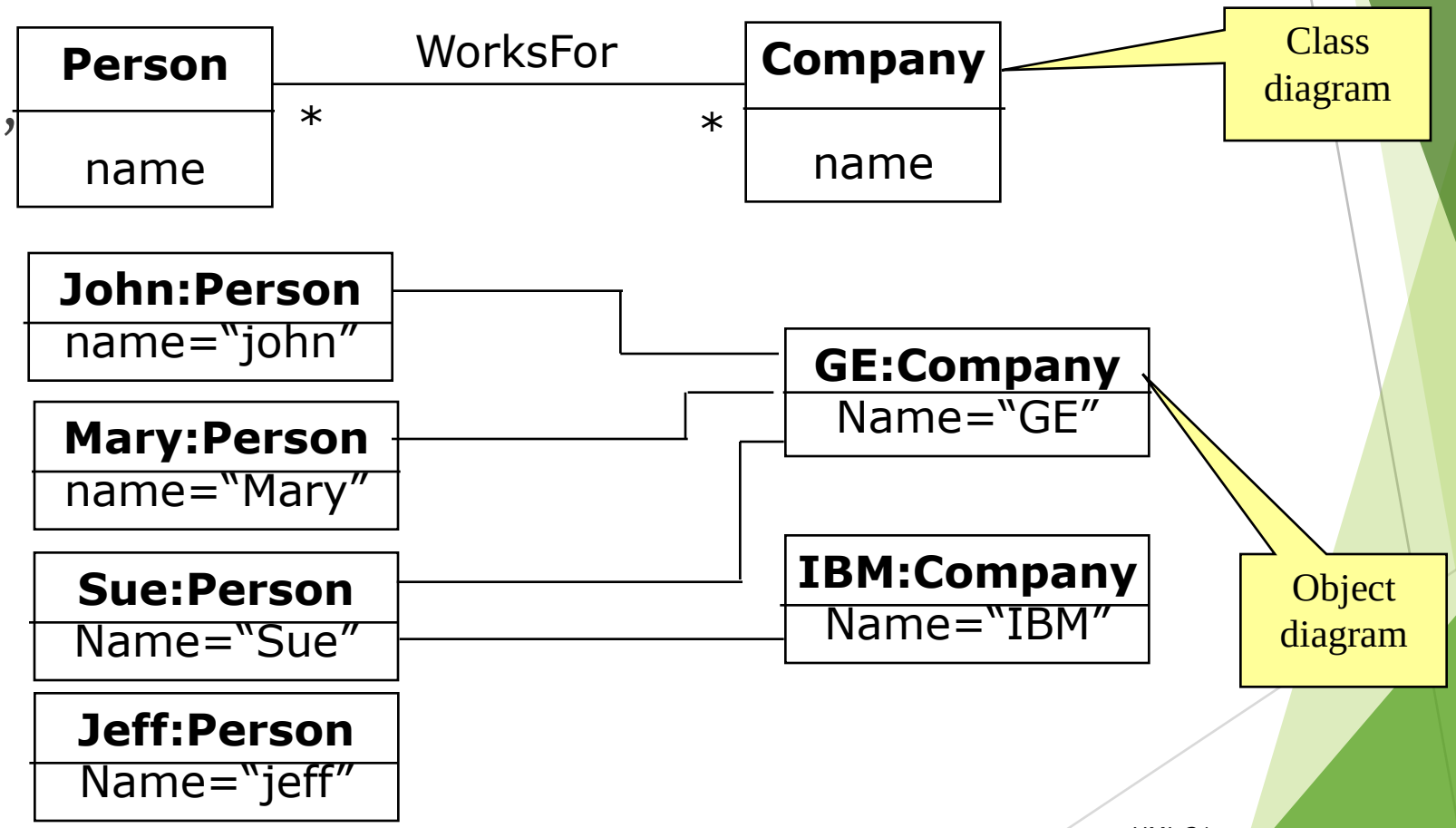
- A link is a physical or conceptual connection among objects.
- Most link relate two objects but some links relate three or more objects.
- e.g. Smith works for simplex company.
- A link is an instant of an association.
- An Association is a description of a group of links with common structure and common semantics.

Link and Association continue…

- An association connects related classes and it is also denoted by a line. (with multiple line segments)
- It is good to arrange the classes in an association from left to right ,if possible.
- The associations and links often appears as verbs in problem statements.

Link and Association continue…

→ For example,

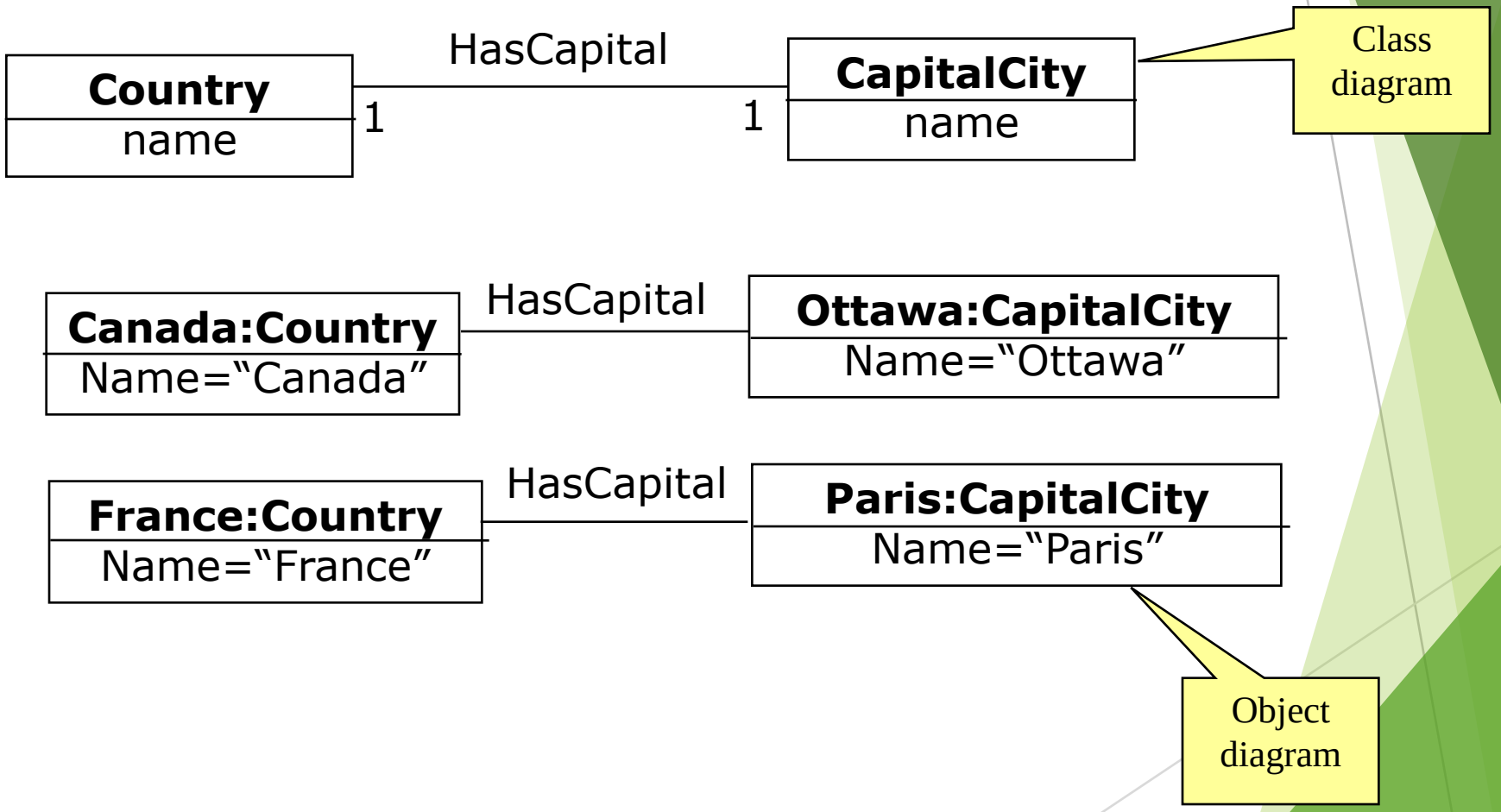


Multiplicity

- Multiplicity specifies the number of instances of one class that may relate to a single instance of an associated class.
- UML diagrams explicitly list multiplicity at the end of the association lines.
- Multiplicity Indicators

Exactly one	1
Zero or more (unlimited)	* (0..*)
One or more	1..*
Zero or one (both inclusive)	0..1
Specified range	2..4
Multiple, disjoint ranges	2, 4..6, 8

Multiplicity Continue…

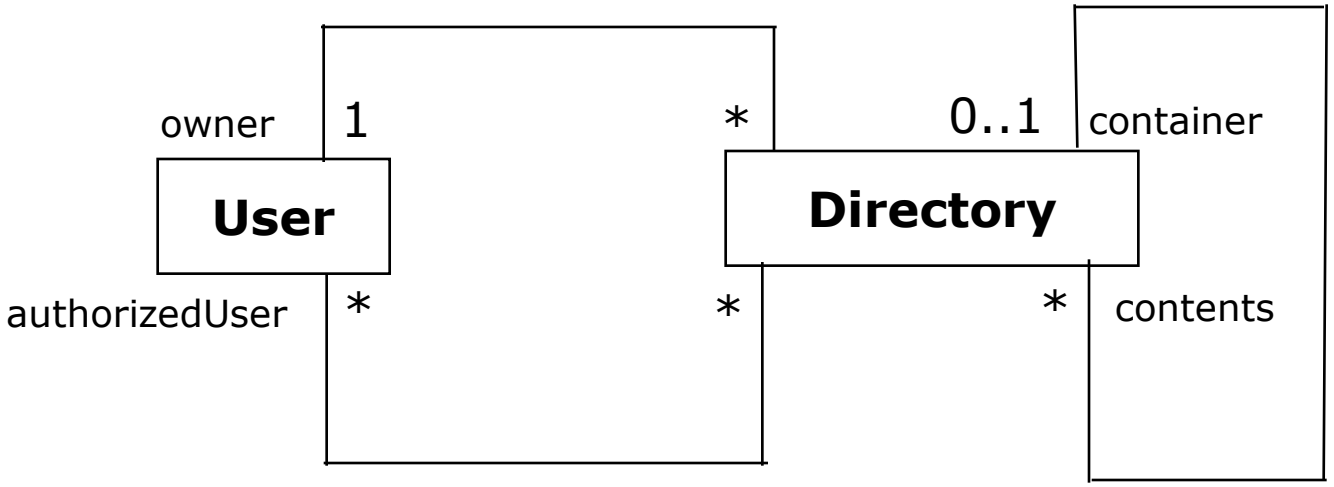


Association End Names

- If we consider one to many association then it has two ends-an end with a multiplicity of “one” and an end with a multiplicity of “many”.
- We can also give the name to both the ends.
- Association end names often appear as nouns in problem descriptions.
- Example

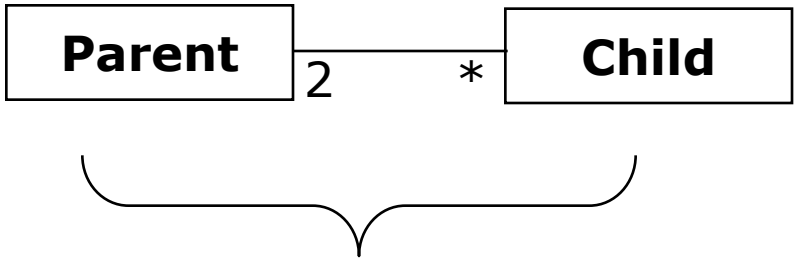
Association End Names Continue…

- Association end names are necessary for associations between two objects of the same class.
- For example container and contents distinguish the two usage of Directory class.

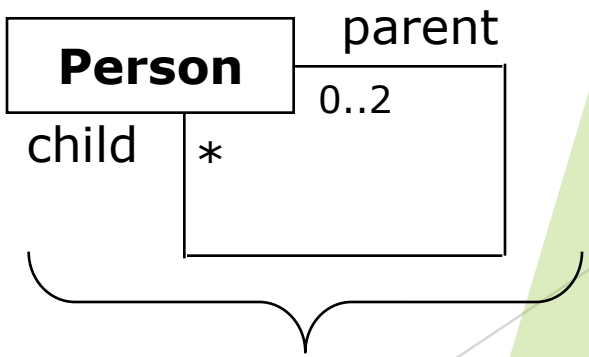


Association End Names Continue…

- Association end names let you unify multiple references to the same class.
- When constructing class diagram you have to use proper association end names not introduce separate class for each reference as shown in below fig.



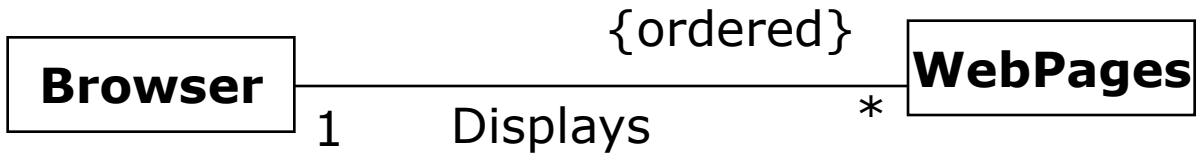
Wrong model



Correct model

Ordering

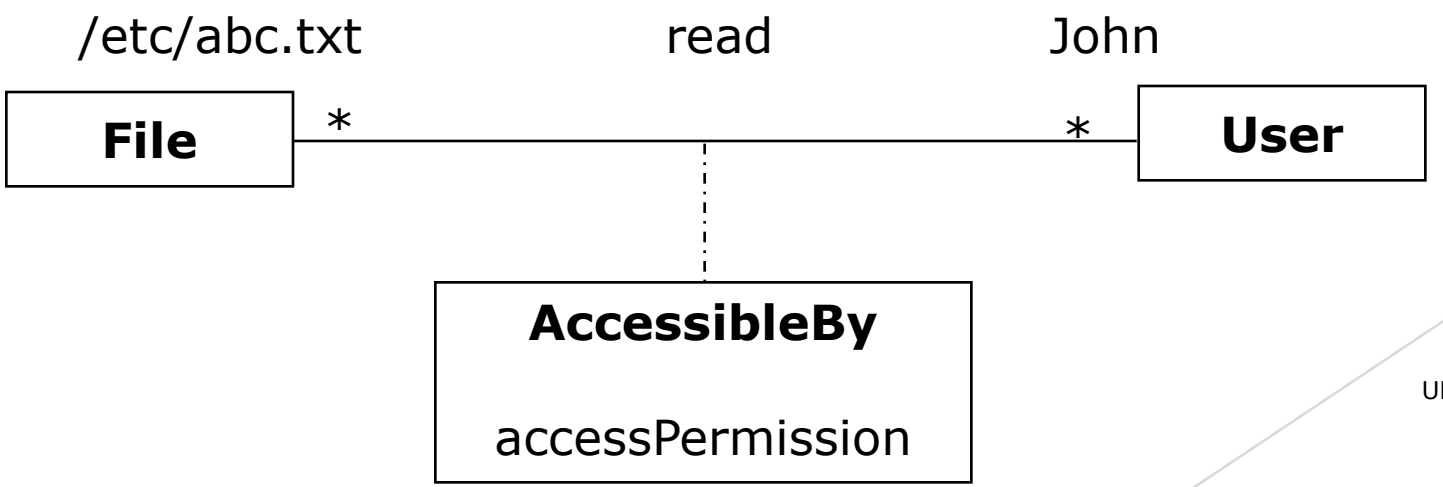
- Often the objects on a “many” association end have no explicit order and we regard them as a set. Sometimes the objects have an explicit order.



- The Web Browser displays the web pages in some specific order (FCFS).Hence {ordered} keyword is used at the association end.

Association Classes

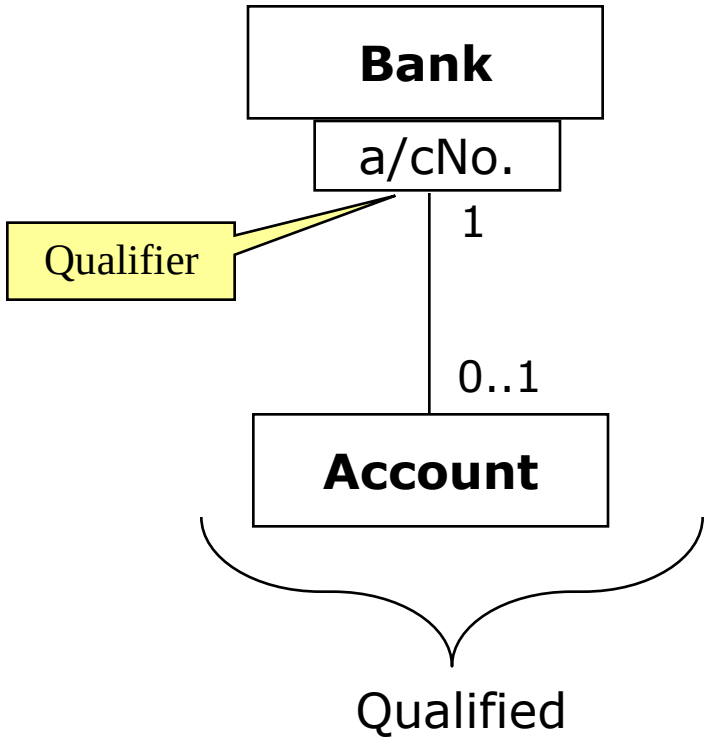
- The abstract class is a class that allows the association to be a class itself.
- When two classes are related with each other by an association link, then the association itself can have attributes and operations
- Hence, association can be represented by a class.
- The UML notation for an association class is a box attached to the association by a dashed line.

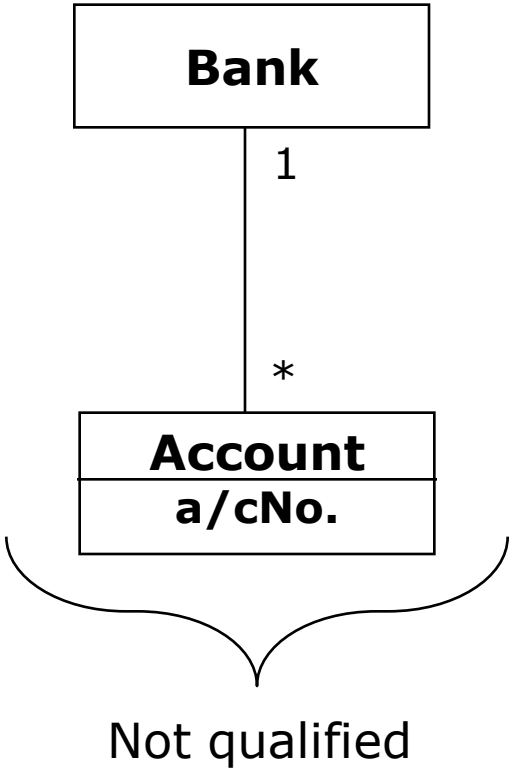


Qualified Associations

- A qualified association is an association in which an attribute called the qualifier disambiguates the objects for a “many” association end.
- possible to define for one-to-many and many-to-many asso.
- A qualifier selects among the target objects, reducing multiplicity from “many” to “one”.
- Example:- A bank services multiple accounts.

Qualified Associations Continue…





Generalization and Inheritance

- Generalization is the relationship between a class (super class) and one or more variations of the class (subclasses).

Super Class (Base class)

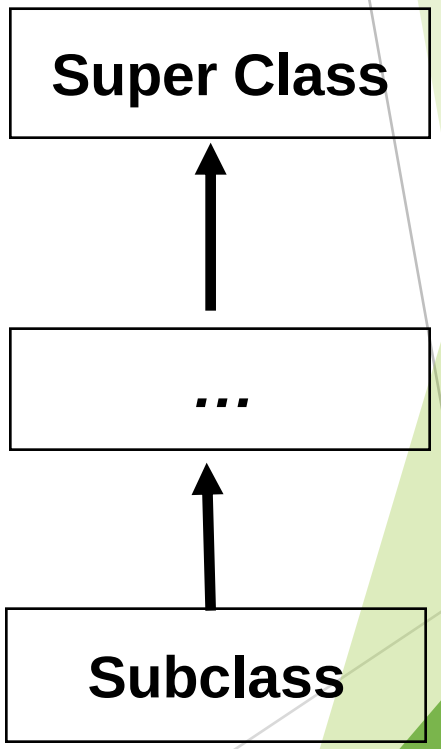
- Provides common functionality and data members

Subclass (Derived class)

- Inherits public and protected members from the super class
- Can extend or change behavior of super class by overriding methods

Overriding

- Subclass may override the behavior of its super class



Generalization and Inheritance Continue…

→A large hollow arrowhead denotes generalization.

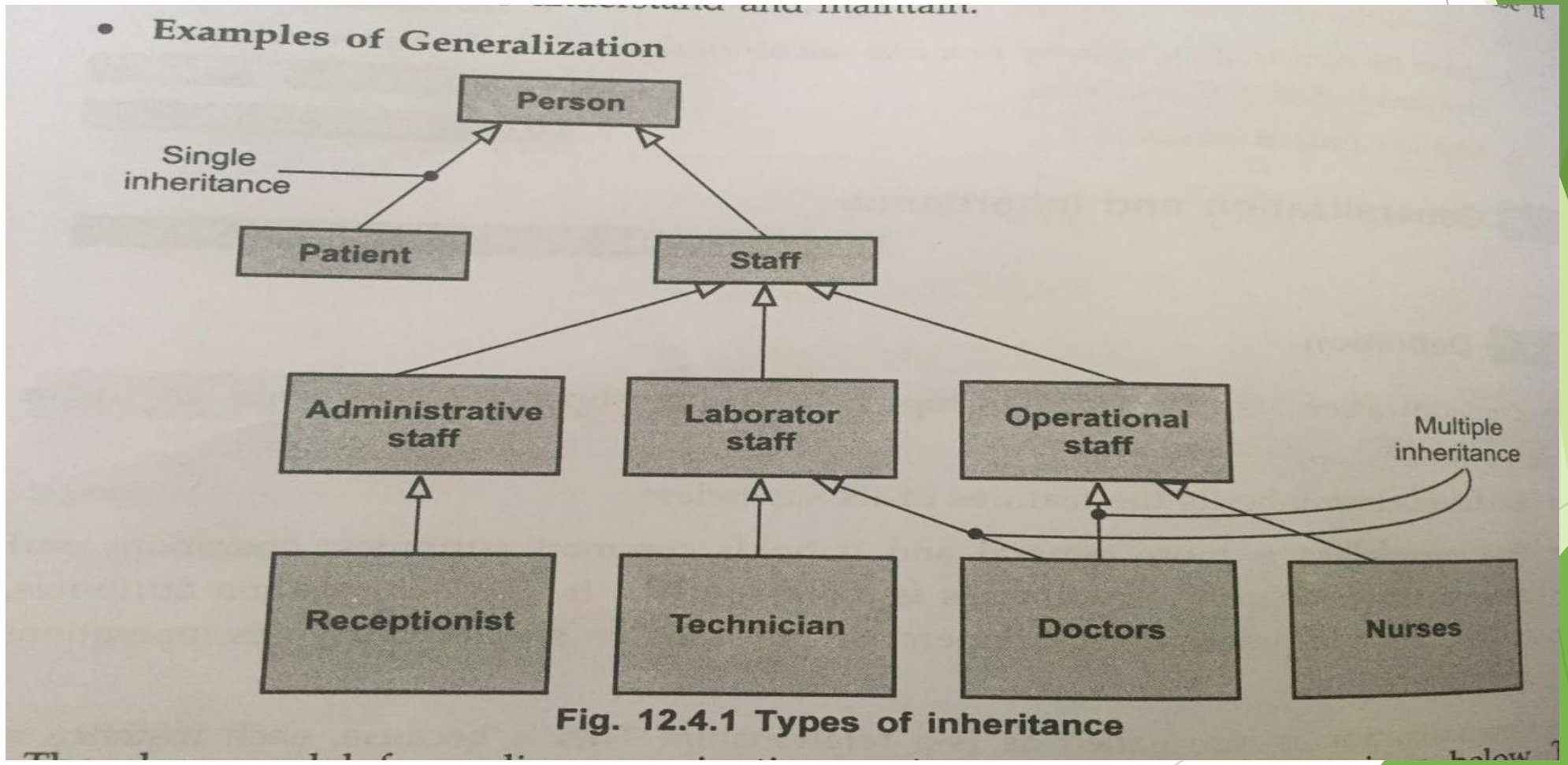


Fig. 12.4.1 Types of inheritance

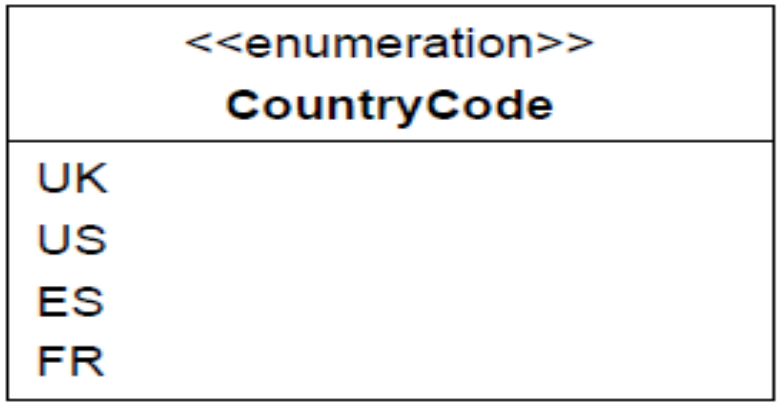
Generalization - advantages

- Polymorphism
- Find common characteristics among classes
- Define hierarchies
- Reuse of the code

ADVANCED CLASS MODELING

Enumerations (enum)

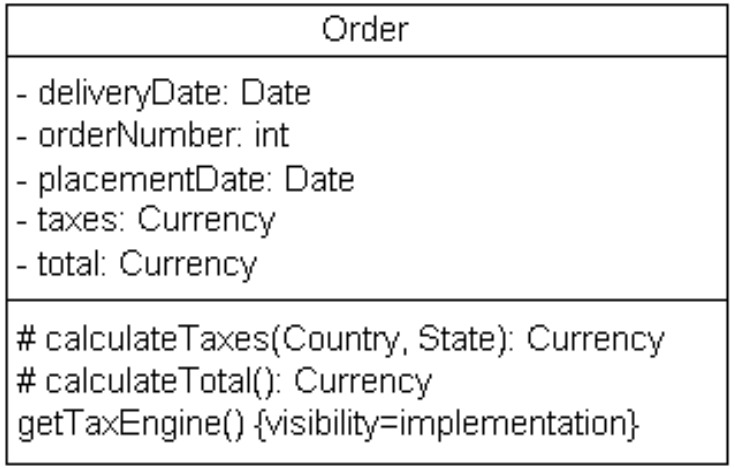
- In Java, an enumeration is a way to have a type that has a defined number of possible values.
- Example country code is represented as an enum. This had the possible values UK, US, ES, FR.
- In UML, we would write it as a class with two compartments. We would add the stereotype <<enumeration>> to the name; you might also see this as <<enum>>.
- The attributes compartment would list the possible values.



Accessibility Operators

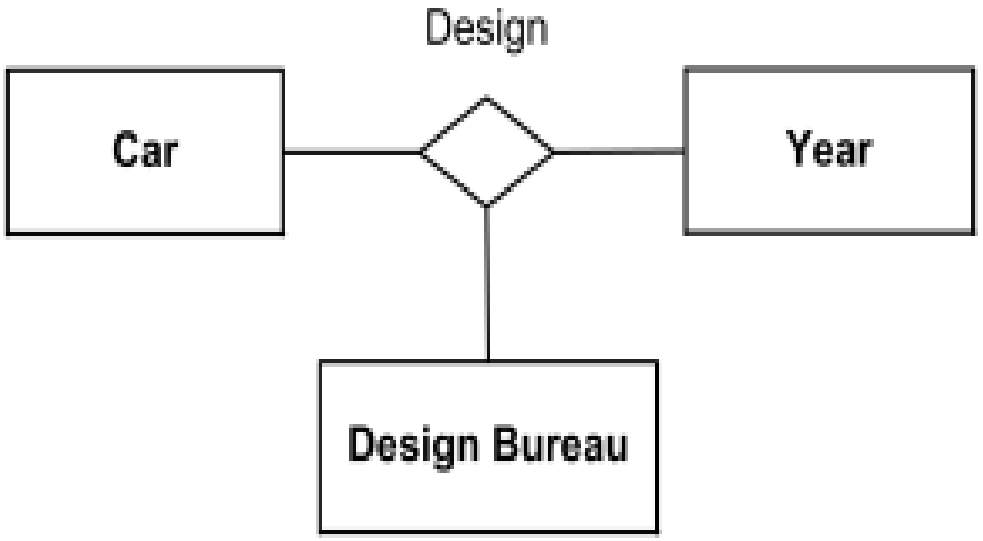
- The class diagram will specify the accessibility for attributes and operations. The most common operators are:
- **+ public**
- **# protected**
- **- private**
- **~package**

Design



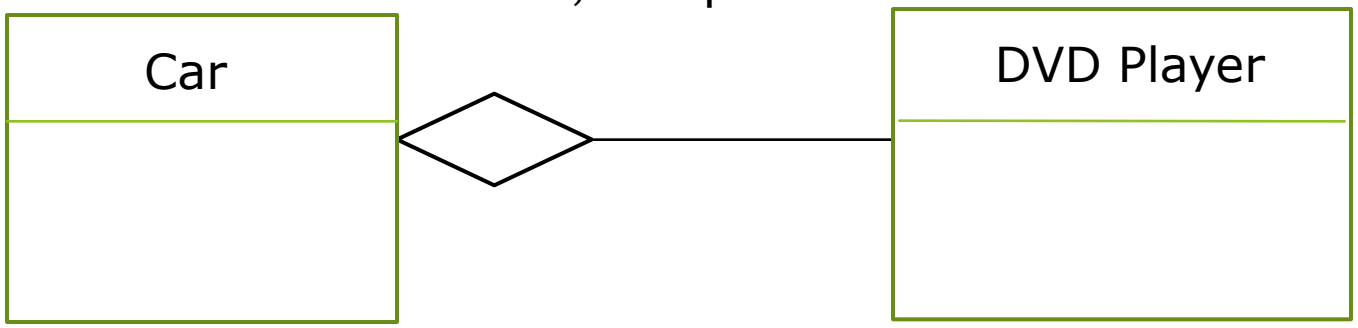
n-Ary Association

- An *n-Ary Association* element is used to model complex relationships between three or more classes.
- It is not a commonly-employed device, but can be used to good effect where there is a dependant relationship between several classes.
- Any n-Ary association may be drawn as a **diamond** with a solid line for each association end connecting the diamond to the classifier that is the end's type.



Aggregation

- Aggregation is a part of association
- It is used to represent whole-part relationship. It is normally posses has-a relationship
- If two objects are tightly bound by a part-whole relationship, it is an **aggregation**.
- If two objects are usually considered as independent, even though they may often be linked, it is an **association.**
- Aggregation is drawn like association, except a small diamond indicates the assembly end.



- Here, DVD player and car are the two classes that can be associated by aggregate relationship.
- Here, Car can exists without DVD player and DVD player can be exists without car.

Aggregation

Example: A LawnMower consists of one blade,one engine,many wheels and one deck.

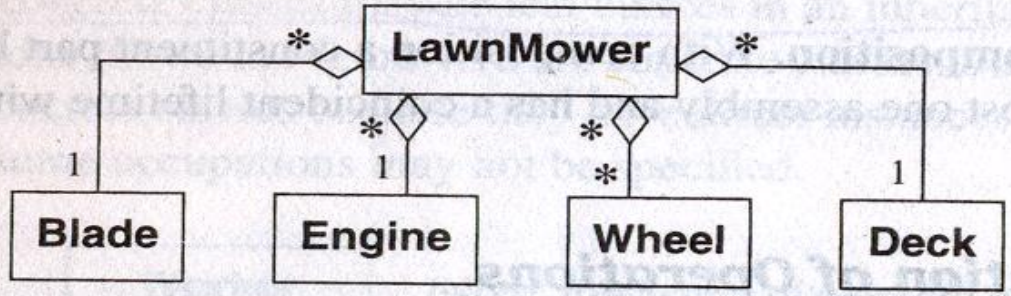


Figure 4.9 **Aggregation**. Aggregation is a kind of association in which an aggregate object is made of constituent parts.

Aggregation Vs. Composition

- The UML has two forms of part-whole relationships: a general form called aggregation and a more restrictive form called composition.
- Composition is a form of aggregation with two additional constraints.

  (1) A constituent part can belong to at most one assembly.

  (2) Once a constituent part has been assigned an assembly, it has a coincident lifetime with the assembly.
- Thus composition implies ownership of the parts by the whole.

Aggregation Vs. Composition Continue…

→The notation for composition is a small solid diamond next to the assembly class.

→In below fig. a company consists of divisions, which in turn consist of departments.

Means company is indirectly a composition of departments.

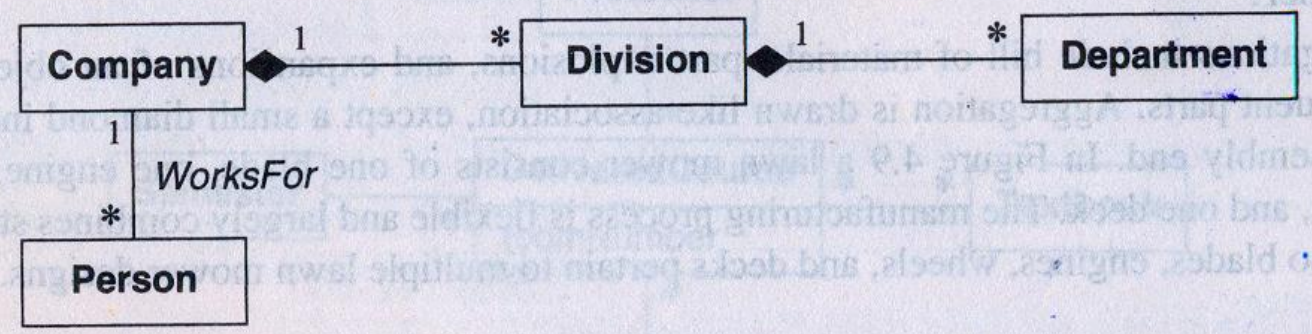
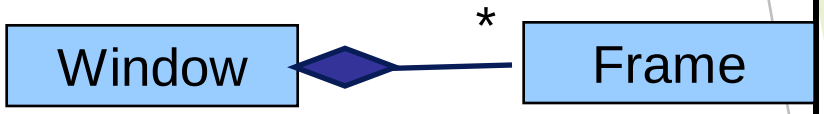


Figure 4.10 Composition. With composition a constituent part belongs to at most one assembly and has a coincident lifetime with the assembly.

Summary of Aggregation Vs. Composition

Aggregation	**Composition**
Part can be shared by several wholes	Part is always a part of a single whole 
Parts can live independently (i.e., whole cardinality can be 0..*)	Parts exist only as part of the whole. When the wall is destroyed, they are destroyed
Whole is not solely responsible for the object	Whole is responsible and should create/destroy the objects

Abstract Classes

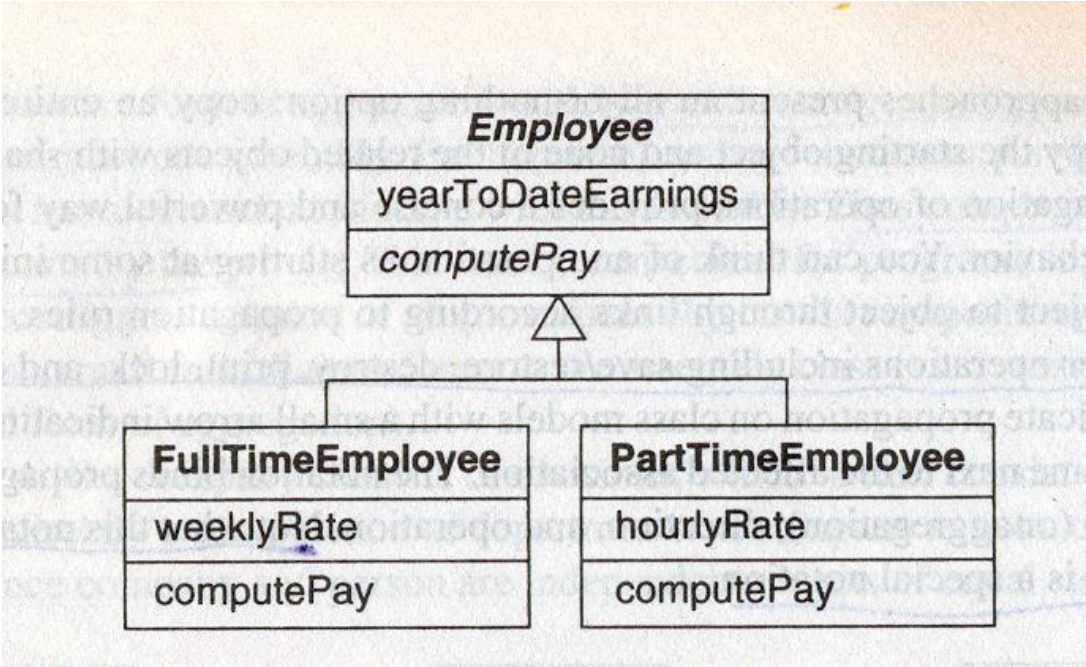
→An **abstract class** is a class that has no direct instances but whose descendant classes have direct instances.

→A **concrete class** is a class that is instantiable that means it can have direct instances.

→Only concrete classes may be leaf classes in an inheritance tree.

→In UML notation an abstract class name is listed in an italic font or by placing the keyword *{abstract}* below or after the name.

→Example:



```
Class A
{
   int a;
   public:
      A() {
         a = 1; }
      virtual void show()
      {          cout <<a;      }
};
Class B: public A
{
   int b;
   public:
      B()
      {          b = 2;       }
      virtual void show()
      {          cout <<b;      }
};
```

```
int main()
{
   A *pA;
   B oB;
   pA = &oB;
   pA→show();
   return 0;
}
```

O/P: 2

Metadata

→Metadata is data that describe other data.For example, a class definition is metadata and models are also metadata.

→Many real world applications have metadata,such as catalogs,dictionary etc.

→Following fig. shows an example of metadata.

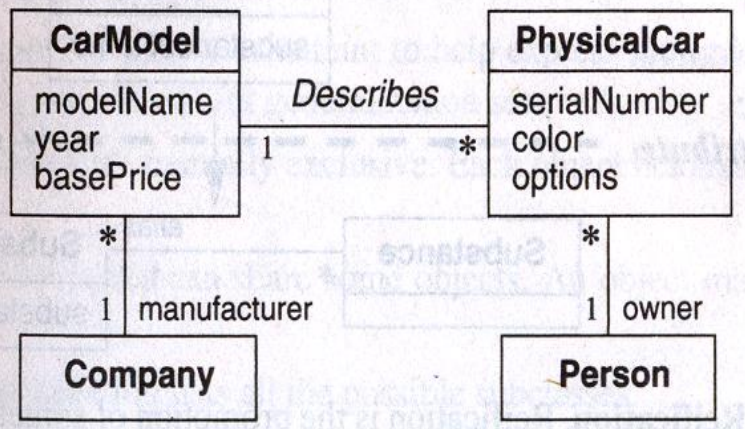


Figure 4.21 Example of metadata. Metadata often arises in applications.

Constraints

- A constraint is a Boolean condition involving model elements such as objects, classes, attributes, links, associations and generalization sets.
- A constraint restricts the values that elements can assume.
- We can define constraints on following:

  - On objects

  - On links

- UML has two notations for constraints.

  - braces { }
  - dog-eared comment box

Constraints on objects

→Following fig. shows several examples of constraints.

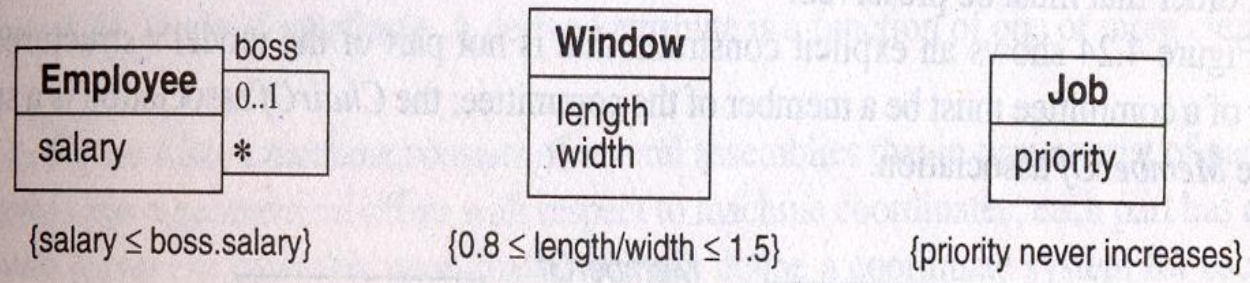


Figure 4.23 Constraints on objects. The structure of a model expresses many constraints, but sometimes it is helpful to add explicit constraints.

→First fig. shows a constraint between two things at the same time.

→Second fig. shows a constraints between attributes of the single object.

→Third fig. shows constraint on the same object over time.

Constraints on Links

→example

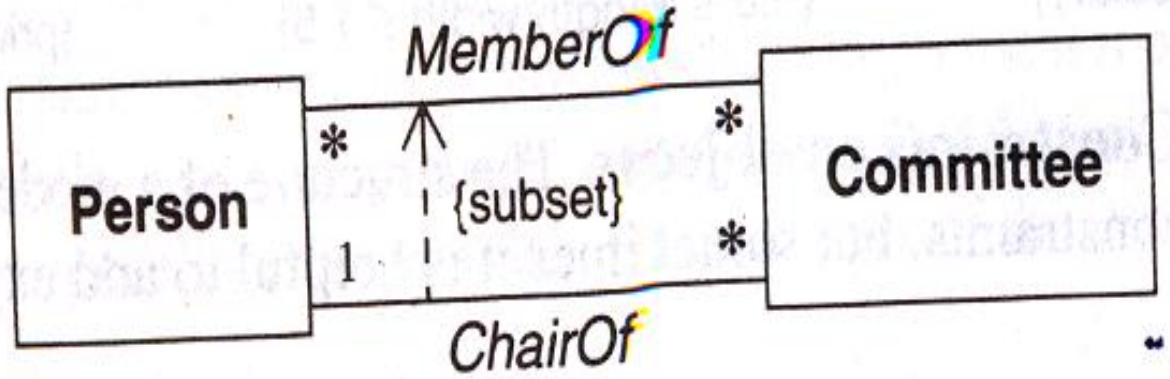
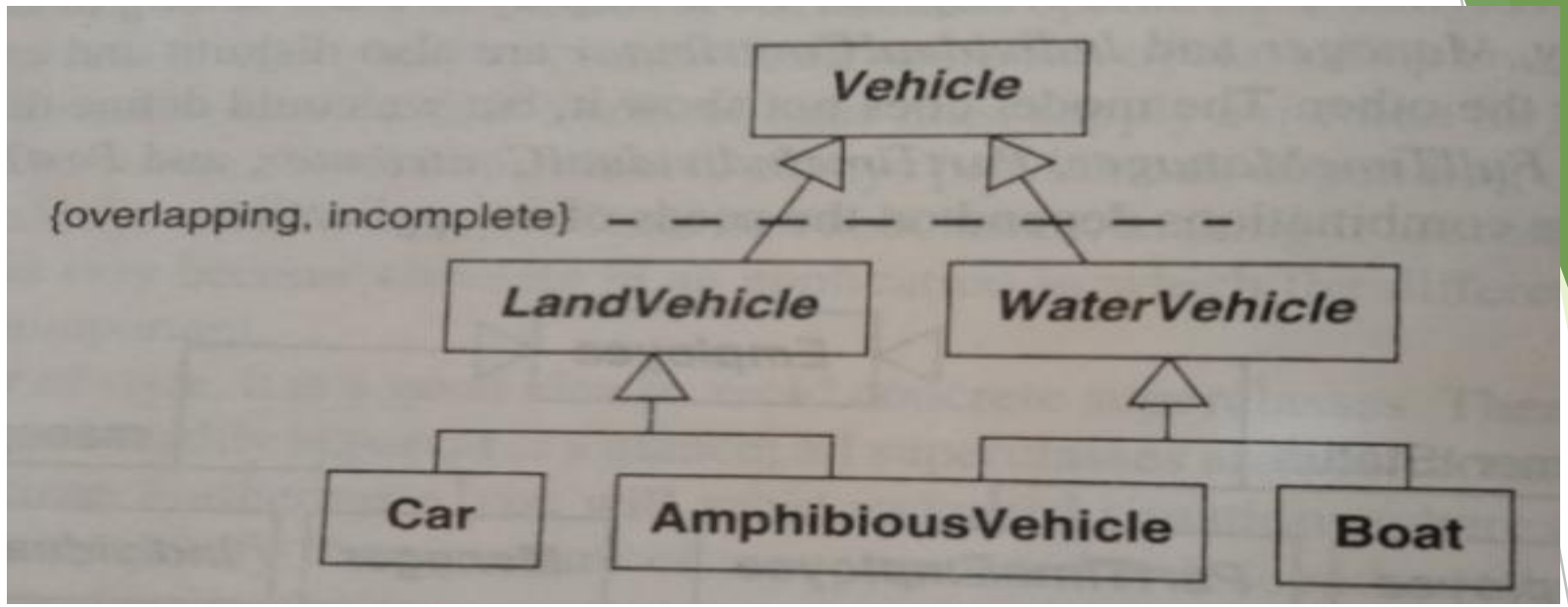
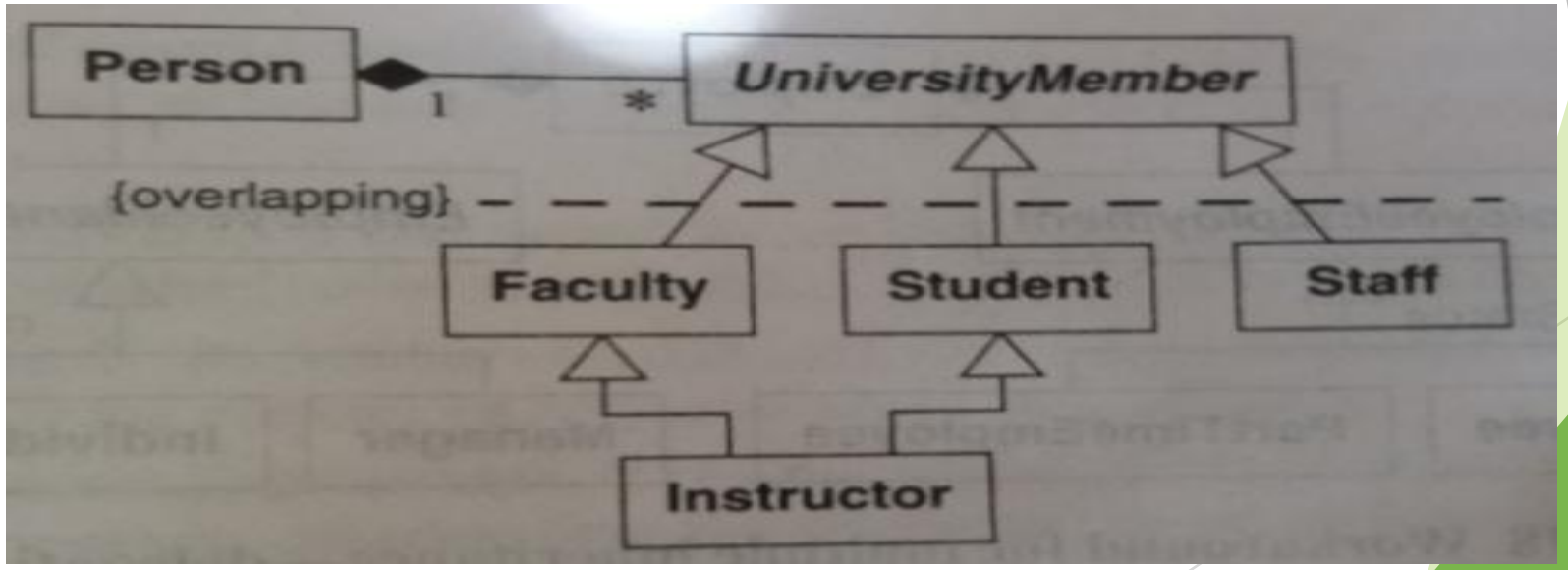


Figure 4.24 Subset constraint between associations.

Constraints on Generalization

- Disjoint
  - Each object belongs to exactly one of the subclasses
- Overlapping
  - An object may belong to more than one subclass
- Complete
  - The generalization lists all the possible subclasses
- Incomplete
  - The generalization may be missing some subclasses

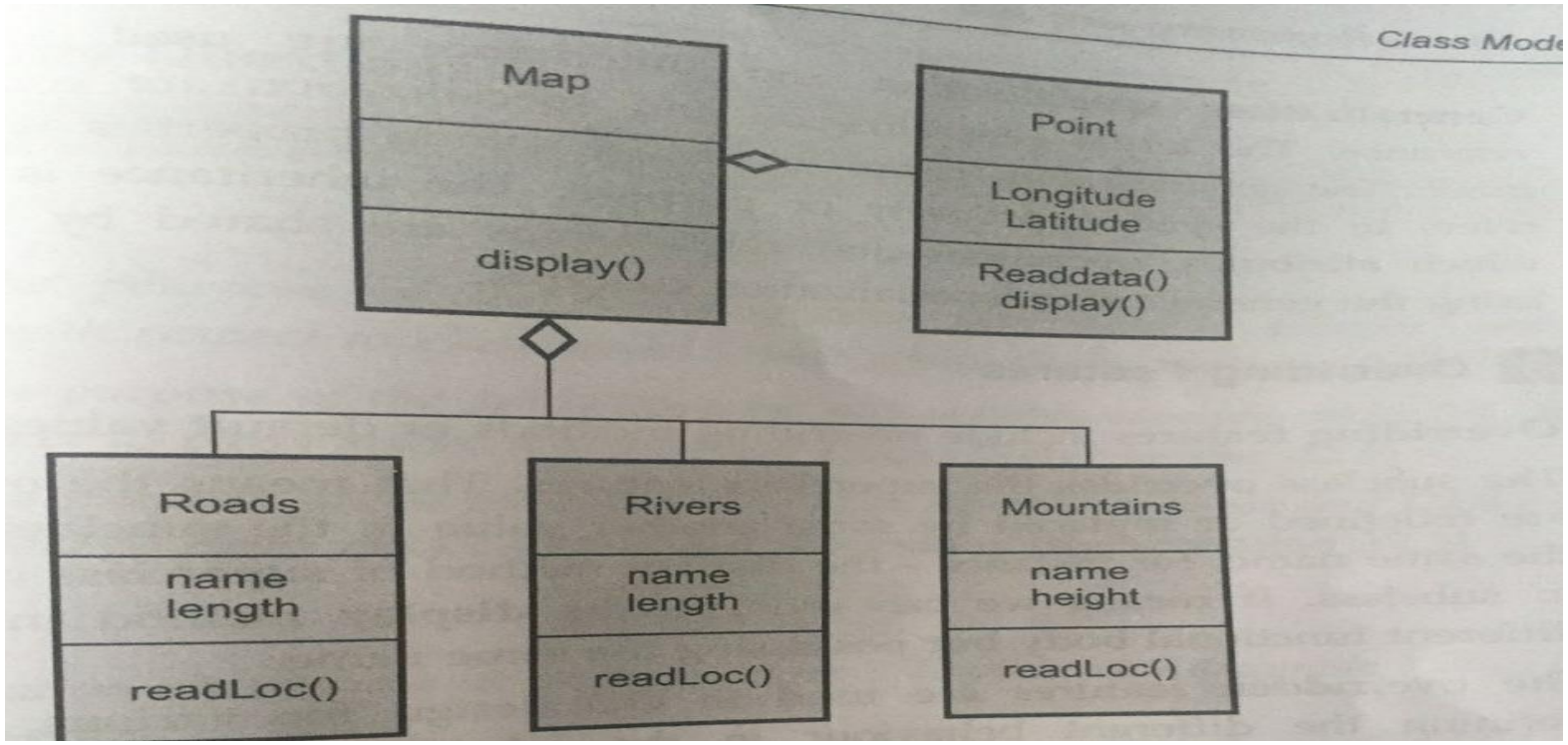

Vehicle
{overlapping, incomplete}
LandVehicle
WaterVehicle
Car
AmphibiousVehicle
Boat


Person
1
*
UniversityMember
{overlapping}
Faculty
Student
Staff
Instructor

Example

Prepare a class model to describe geographical map. Map contains roads, rivers and mountains. All components are described by points representing longitude and latitude.

Solution

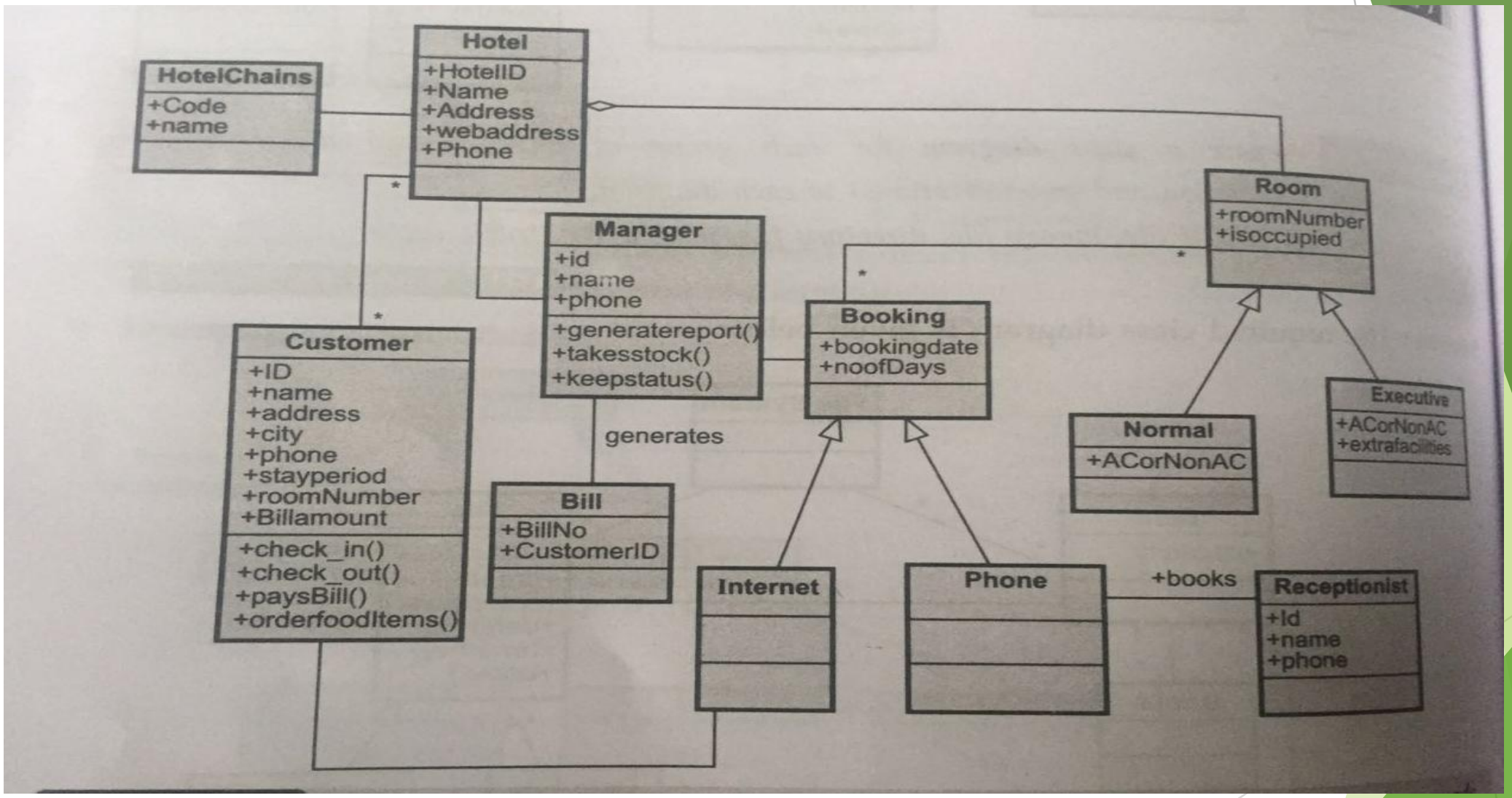

Class Mode
Map
display()
Point
Longitude
Latitude
Readdata()
display()
Roads
name
length
readLoc()
Rivers
name
length
readLoc()
Mountains
name
height
readLoc()

Example

Prepare a class model for the hotel management system.

The system should supports chain of hotels. A hotel contains two categories of rooms: executive and normal, both AC and non-AC. The customers of executive rooms can avail extra facilities like games, swimming, food service in rooms, etc. The booking is possible by internet or by phone. If the booking is through phone, process is done by receptionist, and if booking is done through internet the process is carried out by customer through hotel website. Depending on the number of days customer stays, appropriate bill is generated. The bill also contains amount for transport, food and other facilities enjoyed by the customer along with necessary taxes. The manager should be able to generate reports like list of customers staying in the hotel, list of rooms empty, monthly/yearly income, etc.

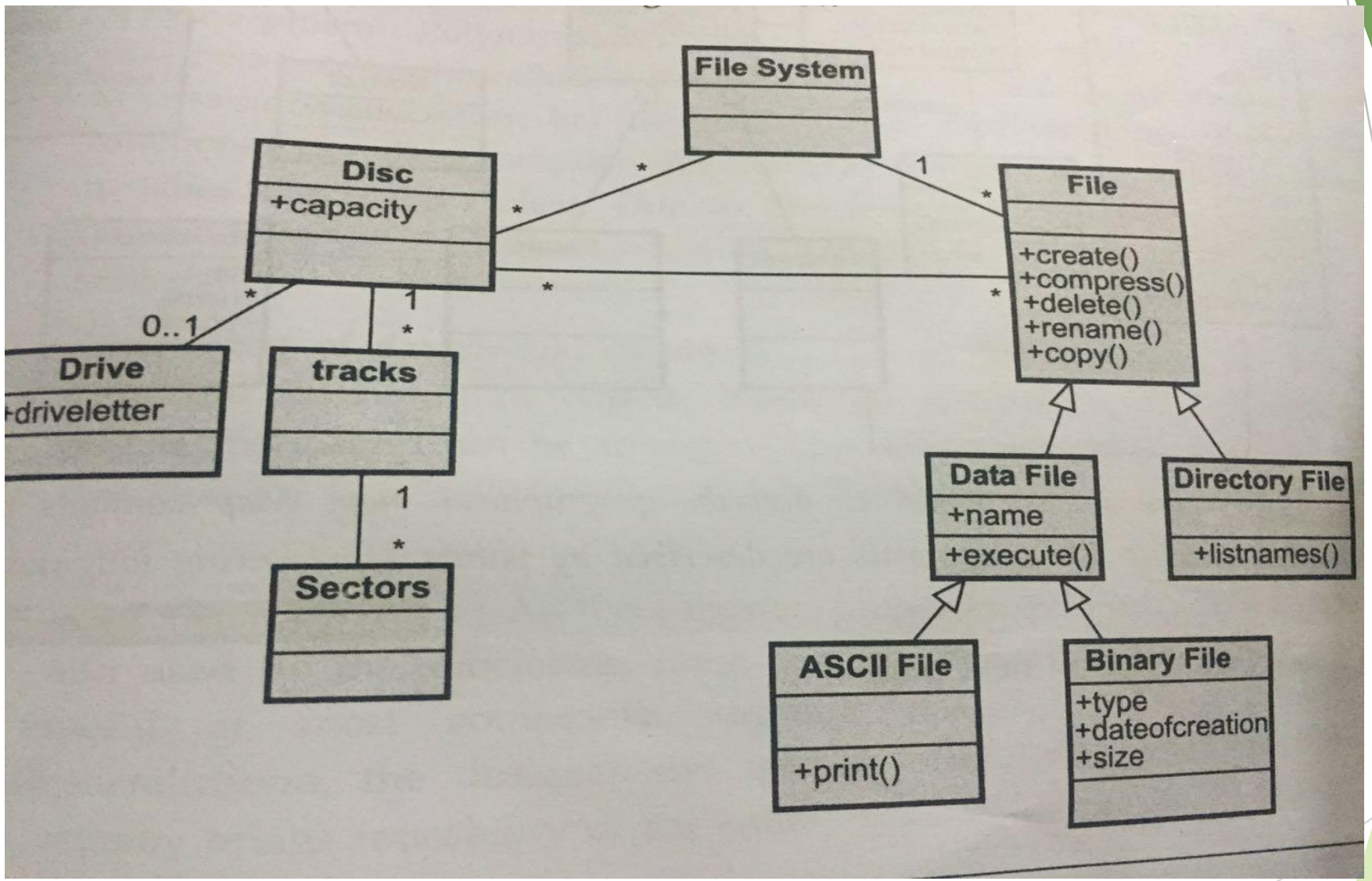
Solution


HotelChains
+Code
+name
Hotel
+HotelID
+Name
+Address
+webaddress
+Phone
Room
+roomNumber
+isoccupied
Manager
+id
+name
+phone
+generatereport()
+takesstock()
+keepstatus()
Booking
+bookingdate
+noofDays
Customer
+ID
+name
+address
+city
+phone
+stayperiod
+roomNumber
+Billamount
+check_in()
+check_out()
+paysBill()
+orderfoodItems()
generates
Bill
+BillNo
+CustomerID
Normal
+ACorNonAC
Executive
+ACorNonAC
+extrafacilities
Internet
Phone
+books
Receptionist
+Id
+name
+phone
*
*
*
*

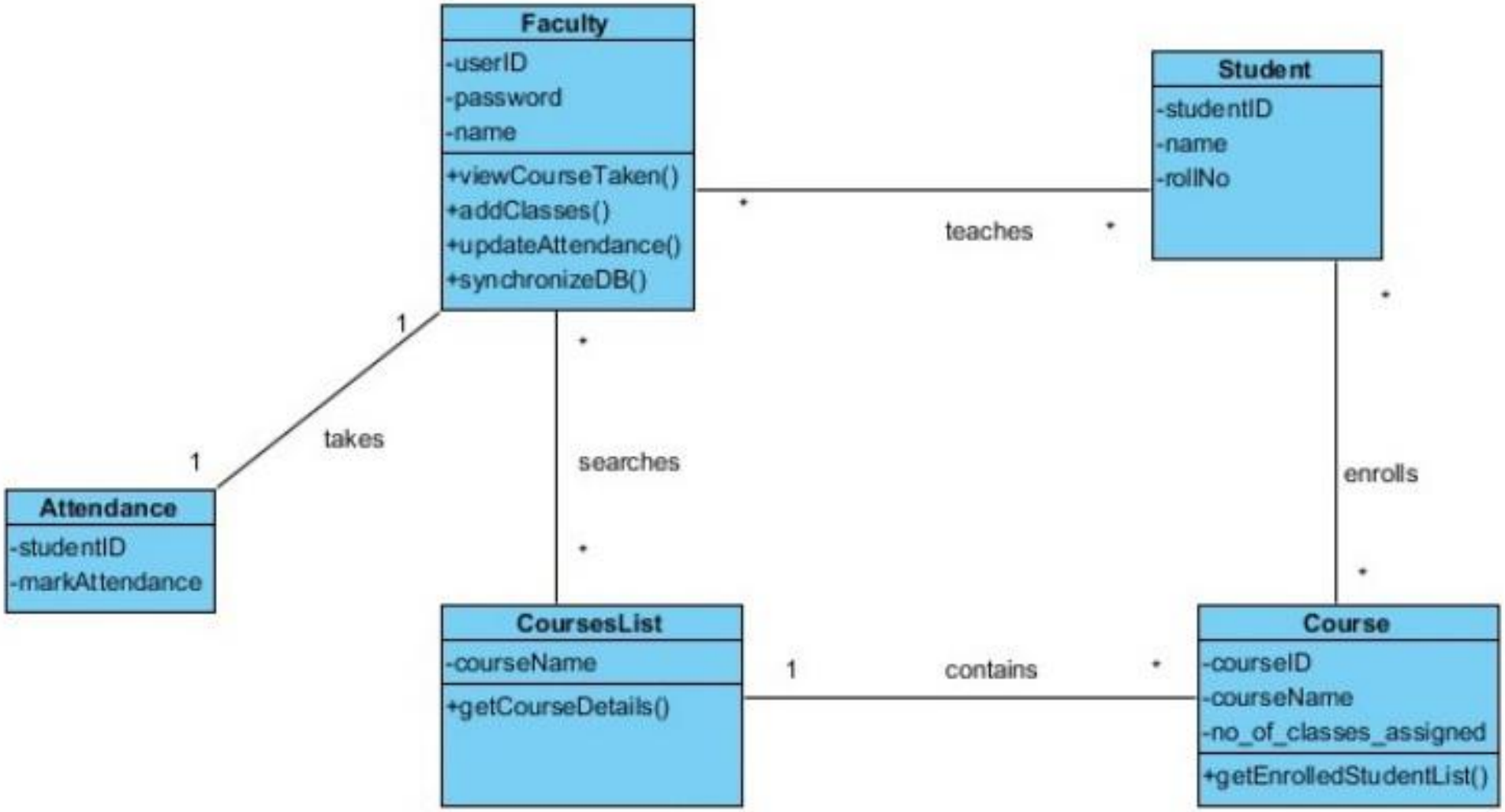
Example

Prepare a class diagram for each group of classes. Add at least 10 relationships (associations and generalizations) to each diagram.

- File system, file, ASCII file, binary file, directory file, disc, drive, track, sector


File System
Disc
+capacity
File
+create()
+compress()
+delete()
+rename()
+copy()
Drive
+driveletter
tracks
Sectors
Data File
+name
+execute()
Directory File
+listnames()
ASCII File
+print()
Binary File
+type
+dateofcreation
+size
0..1
1
*

A sample class diagram: Attendance Management System



State Modeling

Introduction

→We can understand the system by first examining its static structure (class model).

→Then we should examine changes to the objects and their relationship over time and it is given by **state model.**

→The state model describes the sequences of operations that occur in response to external stimuli.

→The state model consists of multiple state diagrams, one for each class with temporal behavior that is important to an application.

Events

- An Event is an occurrence at a point in time, such as user depresses left button of the mouse or flight 123 departs from Chicago.
- Events often correspond to verbs in the past tense like

  - power turned on

  - alarm set

- By definition an event happens instantaneously with regard to the time scale of an application.

Events continue…

- Events include error conditions as well as normal occurrences. For example

  - motor jammed

  - transaction aborted

  - timeout

There are several kinds of events but following are most common events.

  - signal event

  - change event

  - time event

Signal Event

- A **signal** is an explicit one-way transmission of information from one object to another.
- If object want to reply then use the separate signal.
- A **signal event** is the event of sending or receiving a signal.
- We are more concern about the receipt of a signal because it causes effects in the receiving object.
- The UML notation is the keyword signal in guillemets(<<>>) above the signal class name in the top section of a box.

Signal Event Continue…

- For example, UAflight 123 departs from chicago on January 10,1991 is an instance of signal class FlightDeparture.

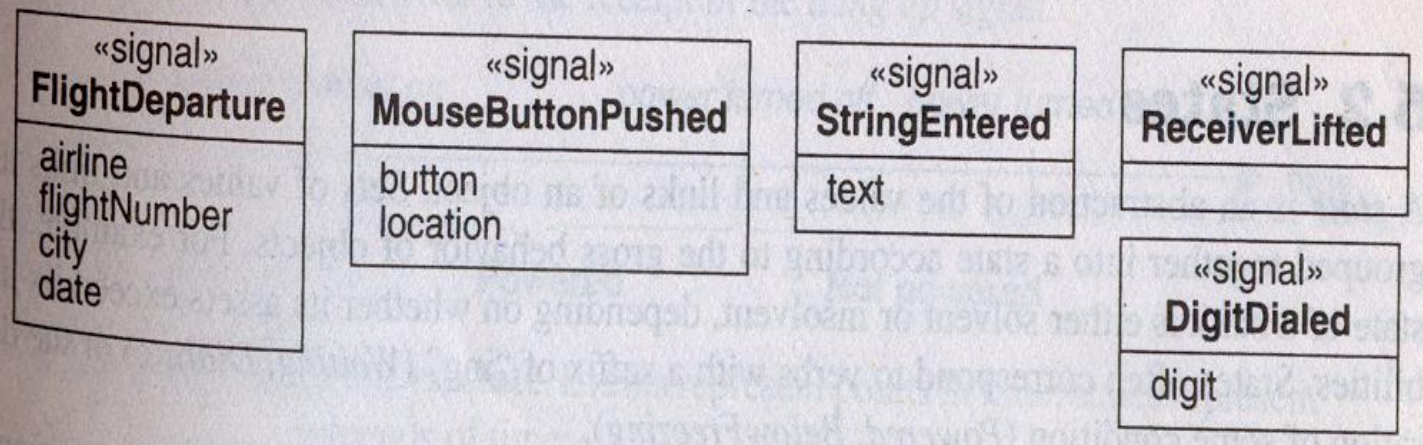


Figure 5.1 Signal classes and attributes. A signal is an explicit one-way transmission of information from one object to another.

Change Event

- A change event is an event that is caused by the satisfaction of a boolean expression.
- The intent of a change event is that the expression is continually tested whenever the expression changes from false to true, the event happens.
- The UML notation for a change event is the keyword “when” followed by a parenthesized boolean expression.
- Examples:

When (room_temp<heating_set_pt)

When (bat_power < lower_limit)

When (tire_pressure < min.pressure)

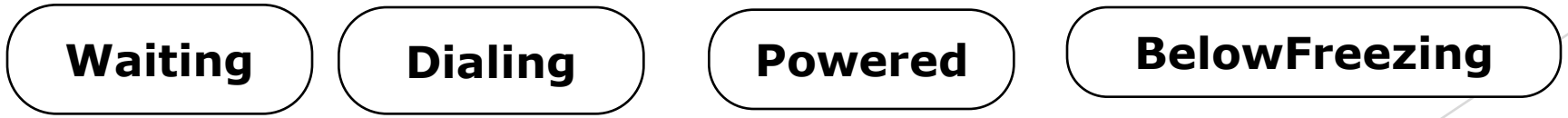
Time Event

- A time event is an event caused by the occurrence of an absolute time or the elapse of a time interval.
- The UML notation for an absolute time is the keyword when followed by a parenthesized expression involving time.
- The notation for a time interval is the keyword after followed by a parenthesized expression that evaluates to a time duration.
- Examples:
  - When (date=Jan 1,2000)
  - After (10 seconds)

States

- A state is an abstraction of the values and links of an object.
- States often correspond to verbs with a suffix of “ing” (Waiting, Dialing) or the duration of some condition (Powered, BelowFreezing).
- UML notation for a state is a rounded box containing an optional state name.

examples:



States Continue…

- Each object can only be in one state at a time.
- Objects may parade through one or more states during their lifetime.
- At a given moment of time, the various objects for a class can exist in a multitude of states.
- A state specifies the response of an object to input events.
- The response may include the invocation of behavior or a change of state.
- For example: if a digit is dialed in state Dial tone, the phone line drops the dial tone and enters state Dialing.

State

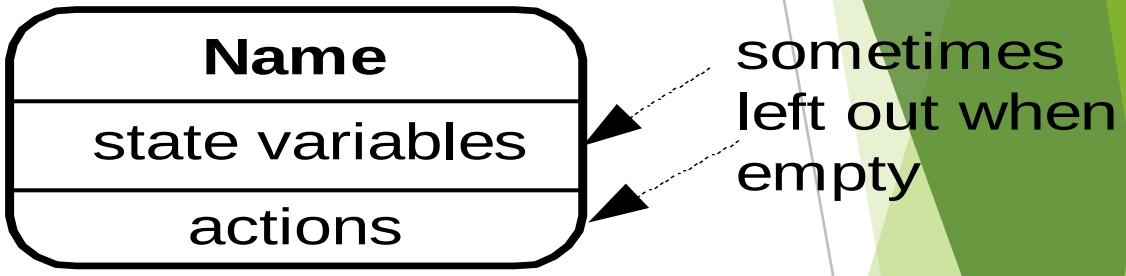
- A state is when a system is:

  Doing something – e.g., heating oven, mixing ingredients, accelerating engine,

  Waiting for something to happen – Waiting for user to enter password, waiting for sensor reading.

Basic UML Syntax

- A state is drawn with a round box, with three compartments for
  - name
  - state variables (if any)
  - actions to be performed



Transitions and conditions

- A transition is an instantaneous change from one state to another.
- For example, when a called phone is answered, the phone line transition from the Ringing state to the Connected state.
- The transition is said to fire upon the change from the source state to the target state.
- The choice of the next state depends on both the original state and the event received.
- An event may cause multiple objects to transition from a conceptual point of view such transition occur concurrently.

Transition

A directed relationship between two states.

Contains parts

- Source state - current state before transition fires.
- Event trigger - external stimulus that has the potential to cause a transition to fire.
- Guard condition - a condition that must be satisfied before a transition can fire.
- Target state - new state after transition fires.

Guard Condition

- A guard condition is a Boolean expression that must be true in order for a transition to occur.
- A guarded transition fires when its event occurs, but only if the guard condition is true.
- For example “when you go out in the morning (event), if the temperature is below freezing (condition), then put on your gloves (next state).”
- A guard condition is checked only once, at the time the occurs and transition fires only if the guard condition is true.
- A guard condition is checked only once while a change event is in effect checked continuously.

Guard Condition Continue…

- The UML notation for a transition is a line from origin state to target state.
- An arrowhead points to the target state.
- The line may consist of several line segments. An event may label the transition and be followed by an optional guard condition in square brackets.
- We italicize the event name and show the condition in normal font.

Guard Condition Continue…

- →Following fig. shows guarded transitions for traffic lights at an intersection.

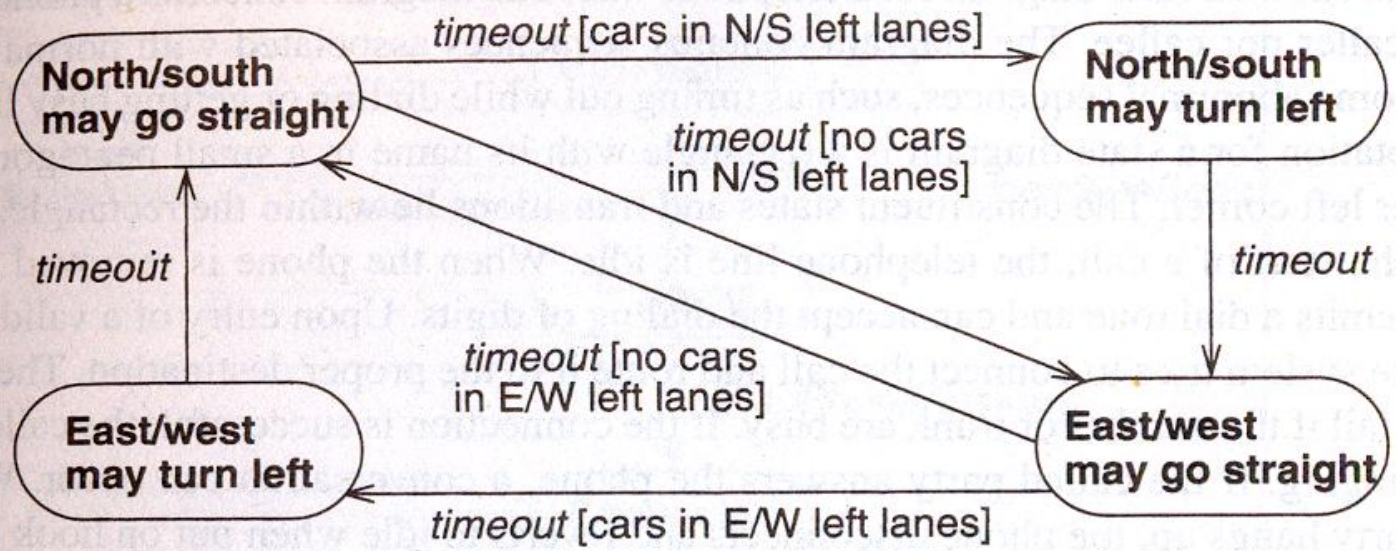
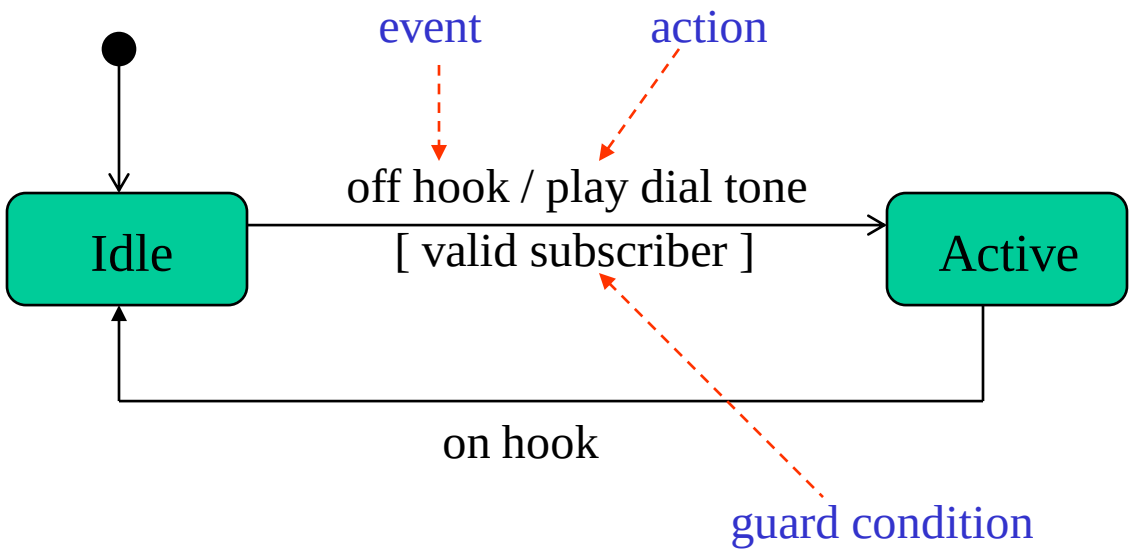


Figure 5.7 Guarded transitions. A transition is an instantaneous change from one state to another. A guard condition is a boolean expression that must be true in order for a transition to occur.


event
action
off hook / play dial tone
[valid subscriber]
Idle
Active
on hook
guard condition

Do -Activities

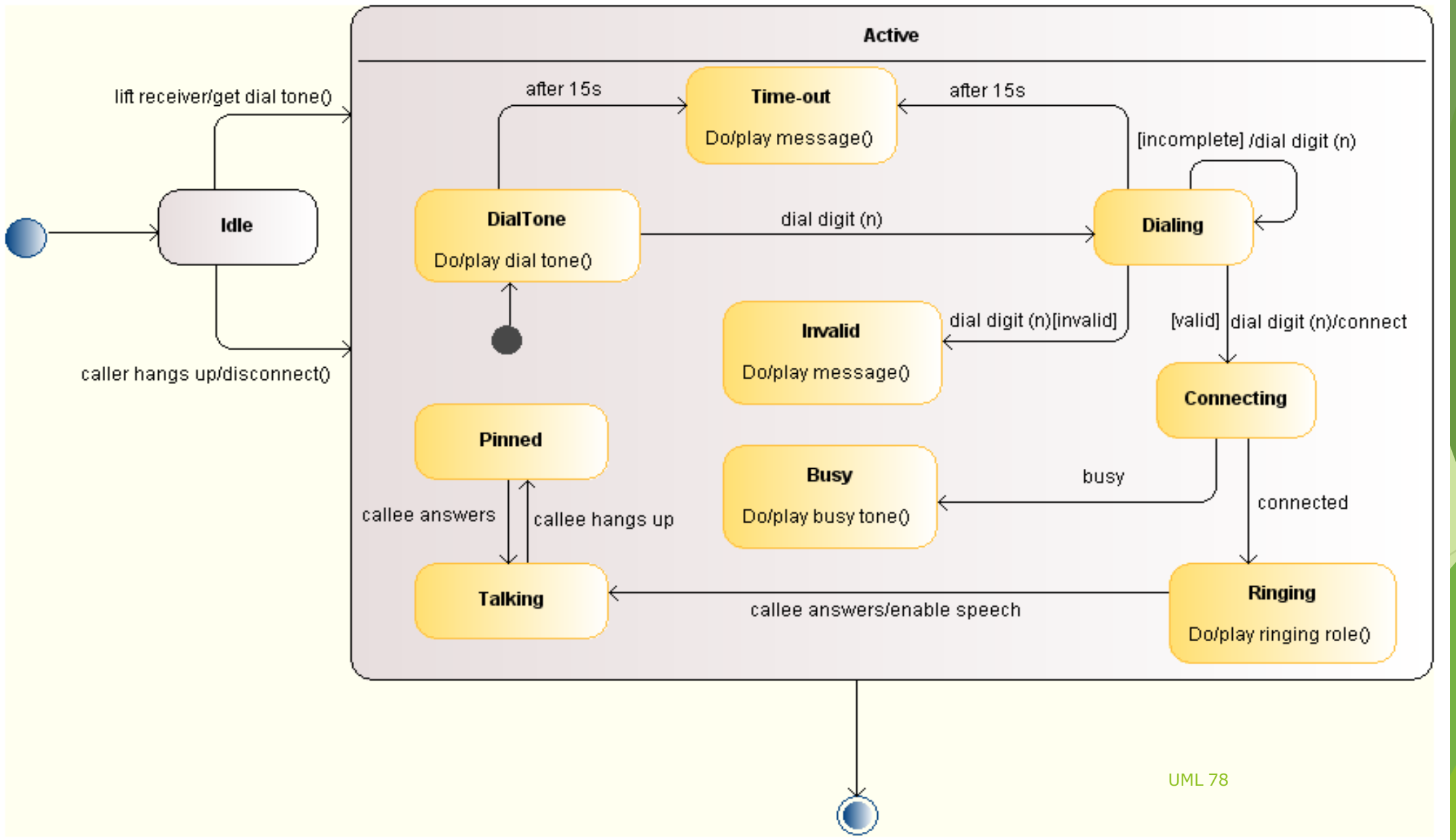
- A do-activity is an activity that continues for extended time.
- It can only occur within a state and can not be attached to a transition.
- For example the warning light may flash during the paper jam state for a copy machine.
- Do-activities include continuous operations such as displaying a picture on a television screen as well as sequential operations that terminate by themselves after an interval of time.

Do -Activities Continue…

- The notation “do /” denotes a do-activity that may be performed for all or part of the duration that an object is in a state.
- A do-activity may be interrupted by an event that is received during its execution.

Paper jam
Do / flash warning light

State Diagram of Telephone System



State Diagram of Elevator

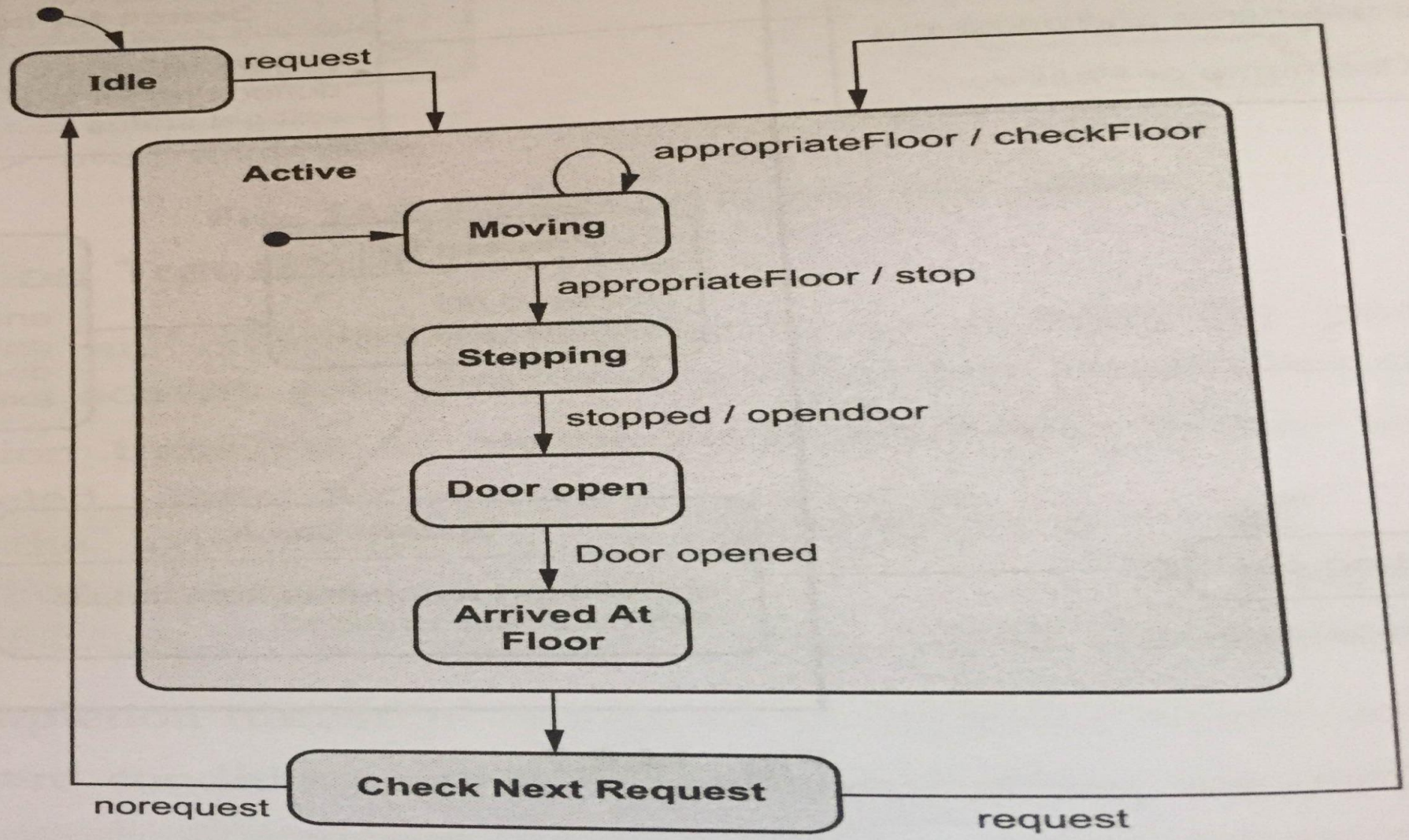


Fig. 14.6.9 Modeling lifetime of object

INTERACTION MODELING

Introduction

→The interaction model is the third leg of the modeling tripod and describes interactions within a system.

→The class model describes the objects in a system and their relationships.

→The interaction model describes how the objects interact.

→In this chapter we will discuss following diagrams:

1)Use Case diagram 2) Sequence diagram 3)Activity diagram

Use Case Models

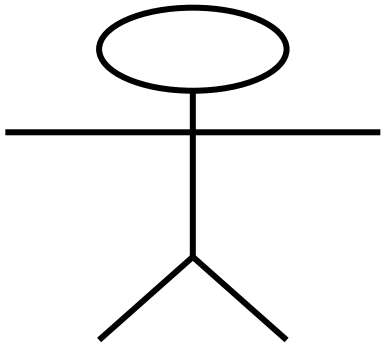
- Use cases describe how a system interacts with outside actors.
- Each use case represents a piece of functionality that a system provides to its users.

Actors:

- An actor is a direct external user of a system.
- An object or set of objects that communicates directly with the system but that is not part of the system.
- Each actor represents those objects that behave in a particular way toward the system.

Use Case Models Continue…

- An actor is directly connected to the system and the object indirectly connected to the system is not called an actor.
- In use case diagram actor is represented by following symbol.



Actor name

Use Cases

- A **use case** is a coherent piece of functionality that a system can provide by interacting with actors.
- For example *a customer* actor can *buy a beverage* from a vending machine.

- Following are the use cases for the vending machine.

  - Buy a beverage
  - Perform scheduled maintenance
  - Make repairs
  - Load items

Use Cases

- Each use case involves one or more actors as well as the system itself.
- The use case buy a beverage involves the customer actor and the use case perform scheduled maintenance involves the repair technician actor.
- In a telephone system the use case make a call involves two actors a caller and a receiver.
- The actors need not all be persons.

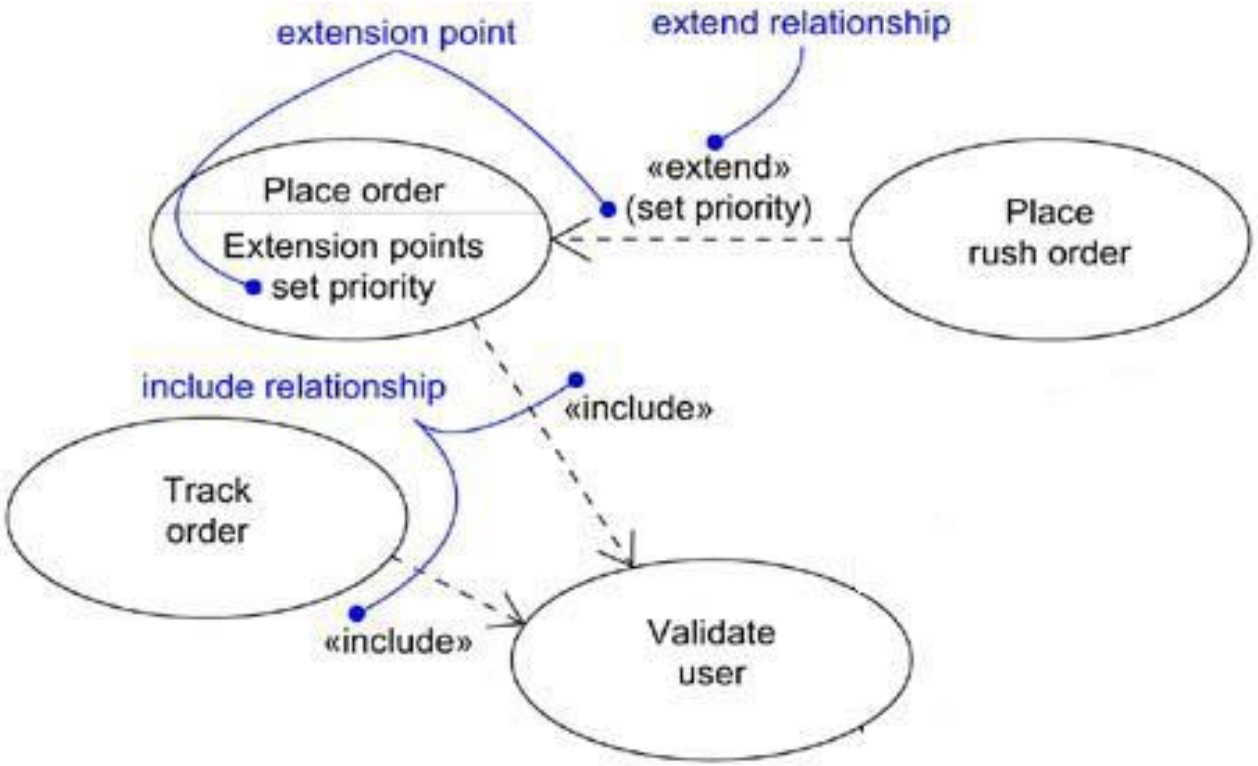
Include Relationship

- The **include** relationship could be used:
- to simplify large use case by splitting it into several use cases,
- to extract **common parts** of the behaviours of two or more use cases.
- A large use case could have some behaviours which might be detached into distinct smaller use cases to be included back into the base use case using the UML **include** relationship. The purpose of this action is modularization of behaviours, making them more manageable.

Extend Relationship

- **Extend** is a **directed relationship** that specifies how and when the behaviour defined in usually supplementary (optional) **extending use case** can be inserted into the **behaviour** defined in the **extended use case**.
- **Extend** relationship is shown as a dashed line with an open arrowhead directed from the **extending use case** to the **extended (base) use case**. The arrow is labelled with the keyword **«extend»**.

Example


extension point
extend relationship
«extend»
(set priority)
Place order
Extension points
set priority
Place
rush order
include relationship
«include»
Track
order
«include»
Validate
user

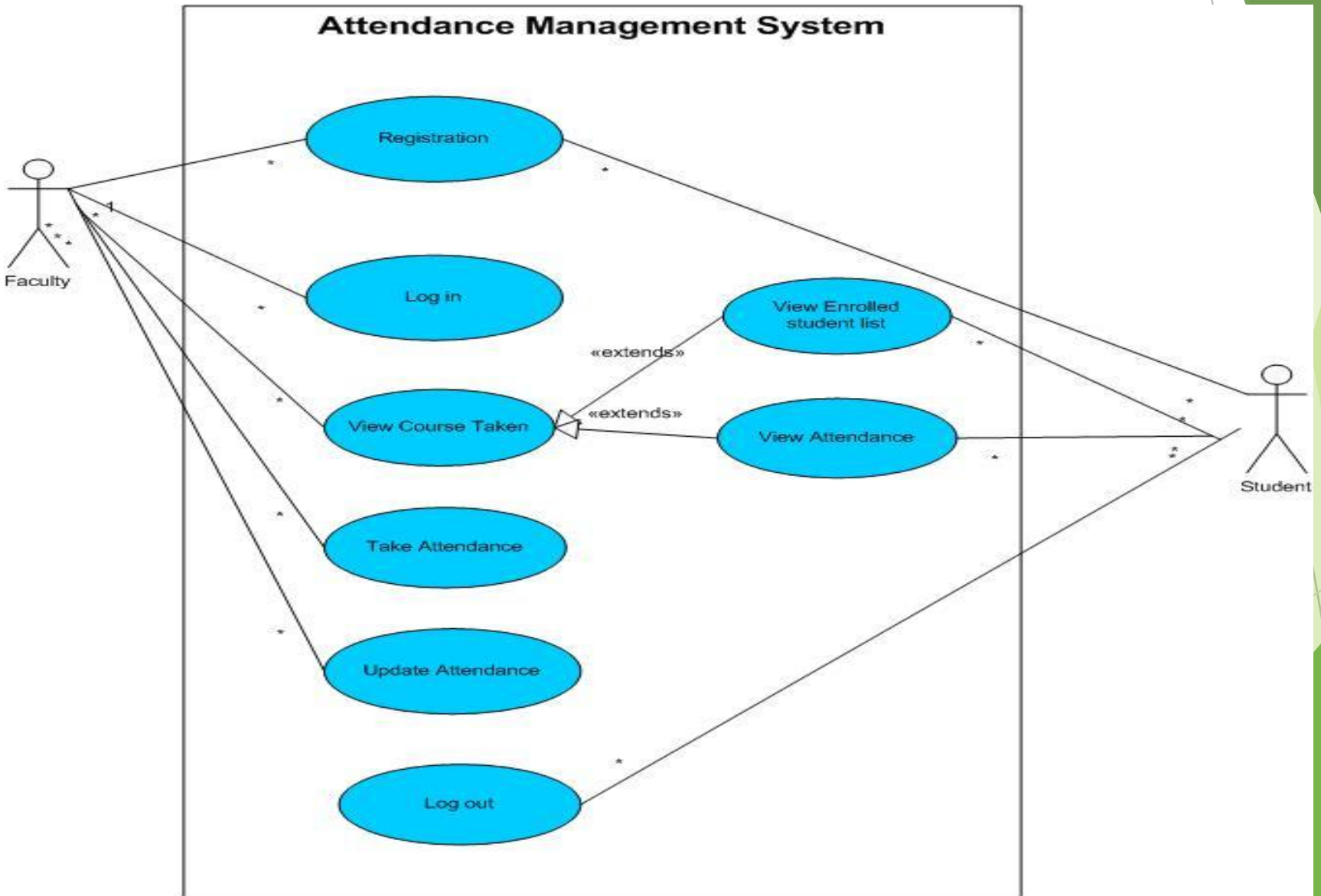
Guidelines for Use case models

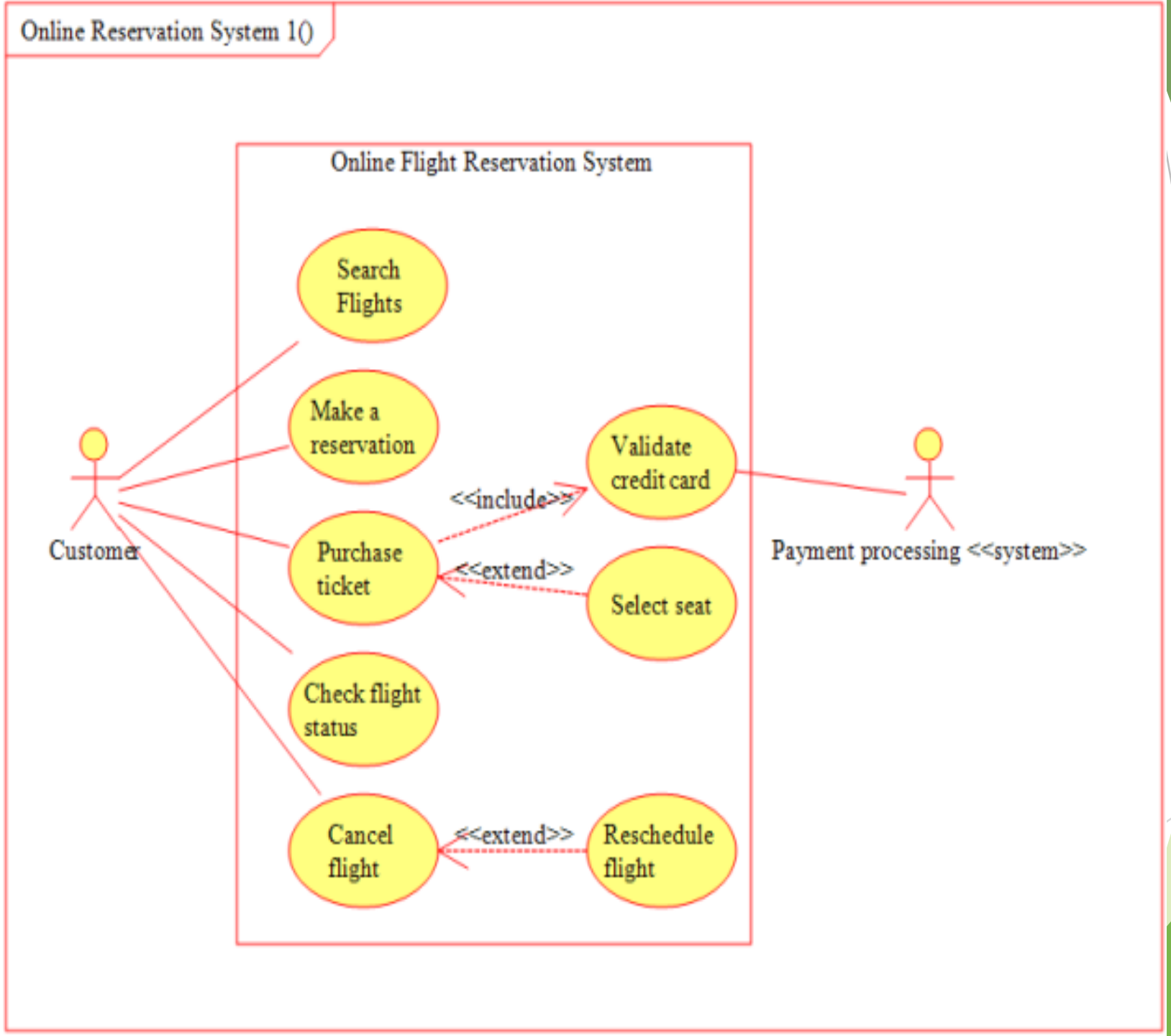
- First determine the system boundary
- Ensure that actors are focused

  -Each actor should have a single purpose. If multiple purposes show them with separate actors.

- Relate use cases and actors

Attendance Management System




Online Reservation System 1()
Online Flight Reservation System
Search Flights
Make a reservation
Purchase ticket
Check flight status
Cancel flight
Validate credit card
Select seat
Reschedule flight
<<include>>
<<extend>>
<<extend>>
Customer
Payment processing <<system>>

Sequence Models

- A **Sequence diagram** is an interaction diagram that shows how processes operate with one another and in what order. A sequence diagram shows object interactions arranged in time sequence.

Key parts of a sequence diag.

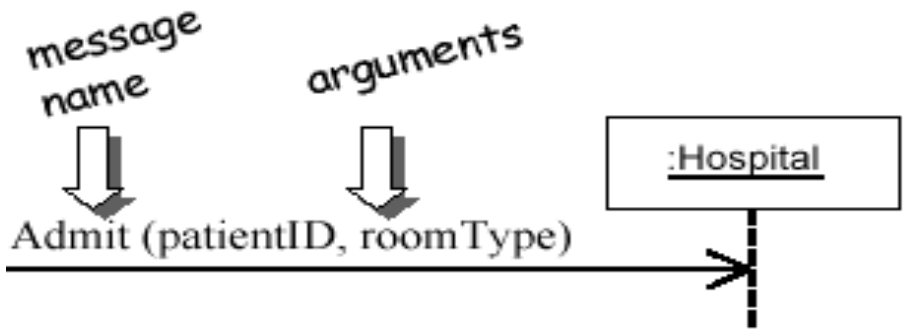
1) participant: an object or entity that acts in the sequence diagram

- sequence diagram starts with an unattached "found message" arrow

Key parts of a sequence diag

2) message: communication between participant objects the axes in a sequence diagram:

- horizontal: which object/participant is acting
- vertical: time (down -> forward in time)
- message (method call) indicated by horizontal arrow to other object
- write message name and arguments above arrow



Key parts of a sequence diag

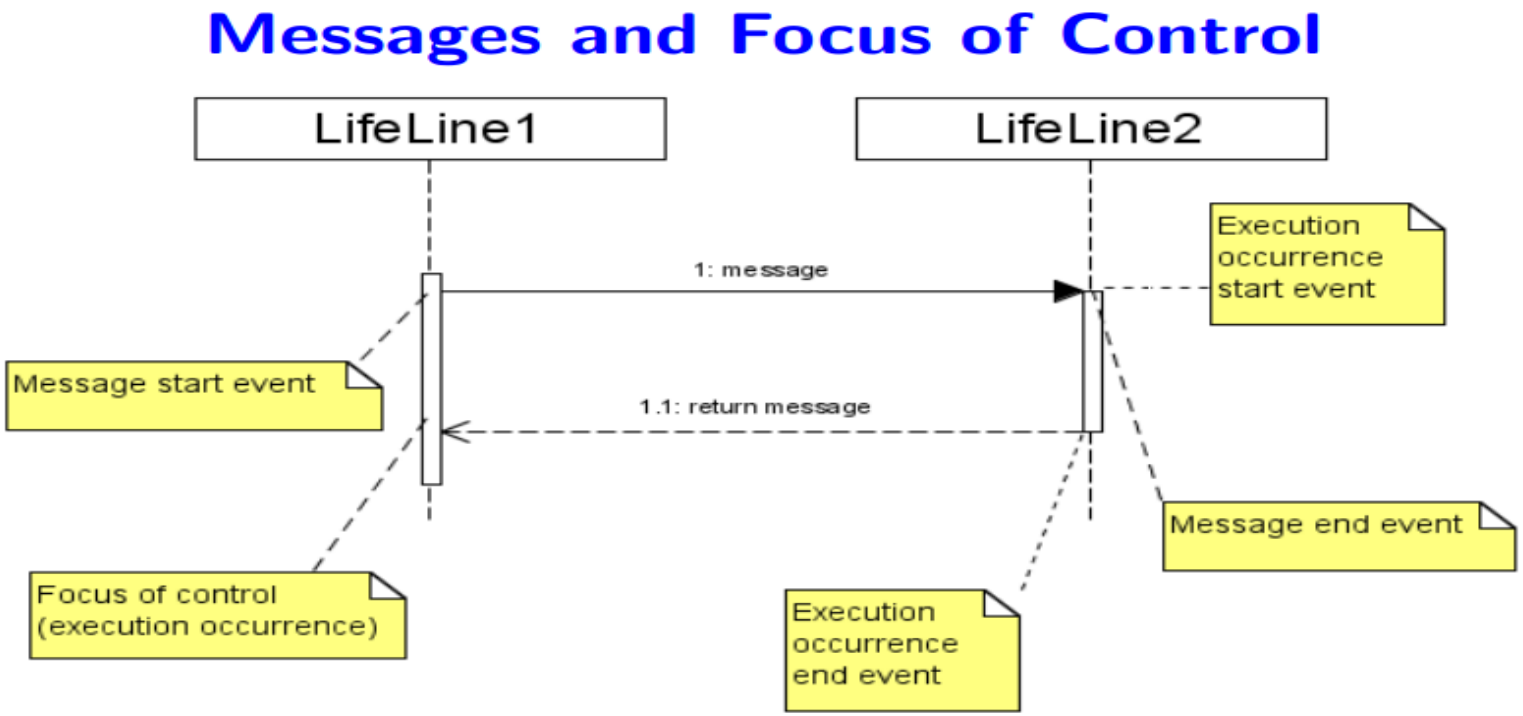
3) Focus of control (execution occurrence):

- An execution occurrence (shown as tall, thin rectangle on a lifeline) represents the period during which an element is performing an operation.
- The top and the bottom of the of the rectangle are aligned with the initiation and the completion time respectively.

Key parts of a sequence diag.

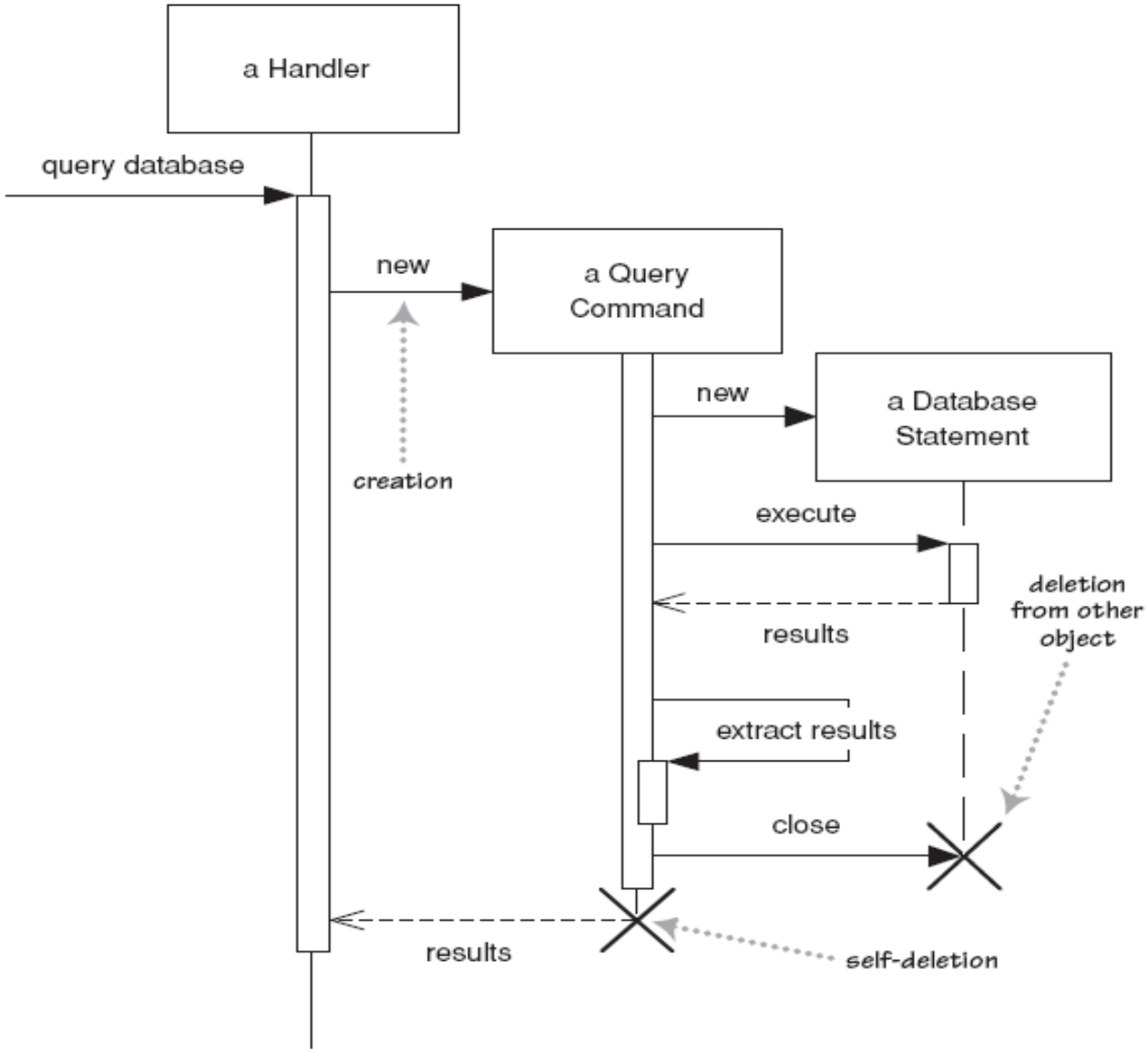
4) Lifeline: squares with object type, optionally preceded by object name and colon

- write object's name if it clarifies the diagram
- object's "life line" represented by dashed vert. line



Lifetime of objects

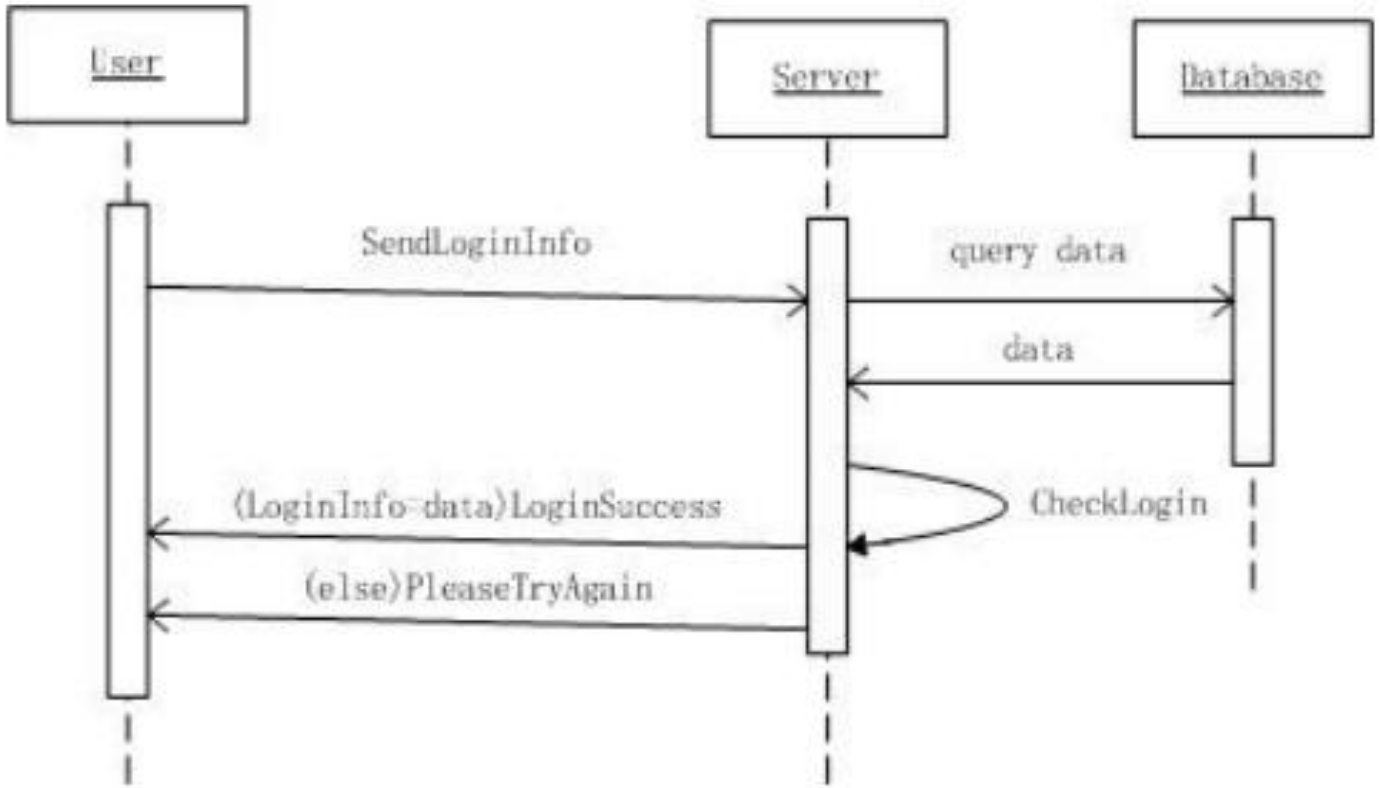
- creation: arrow with 'new' written above it
  - notice that an object created after the start of the scenario appears lower than the others
- deletion: an X at bottom of object's lifeline



Scenarios for AMS

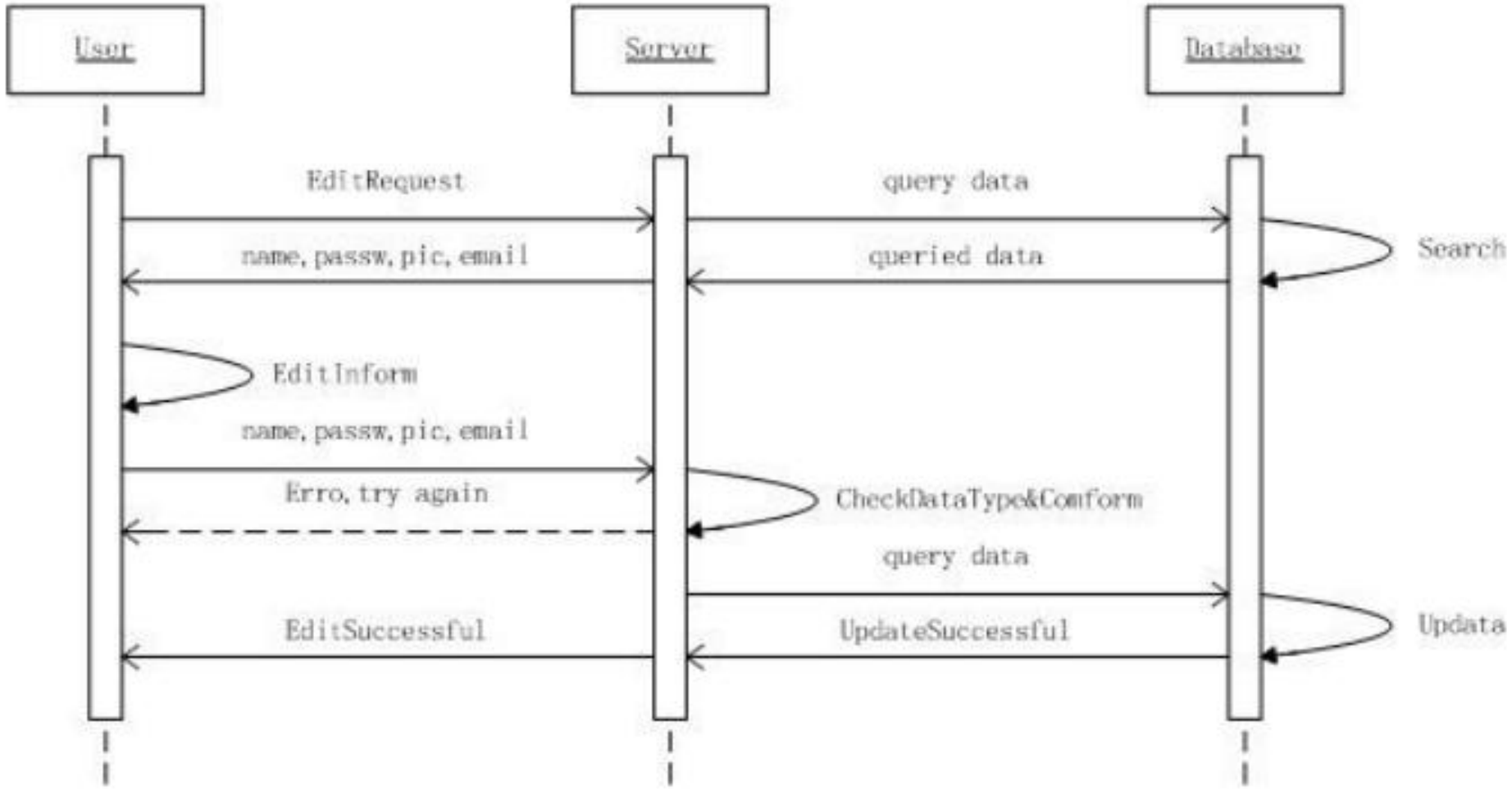
- Each scenario has a sequence of steps.

Scenario 1 : Log in



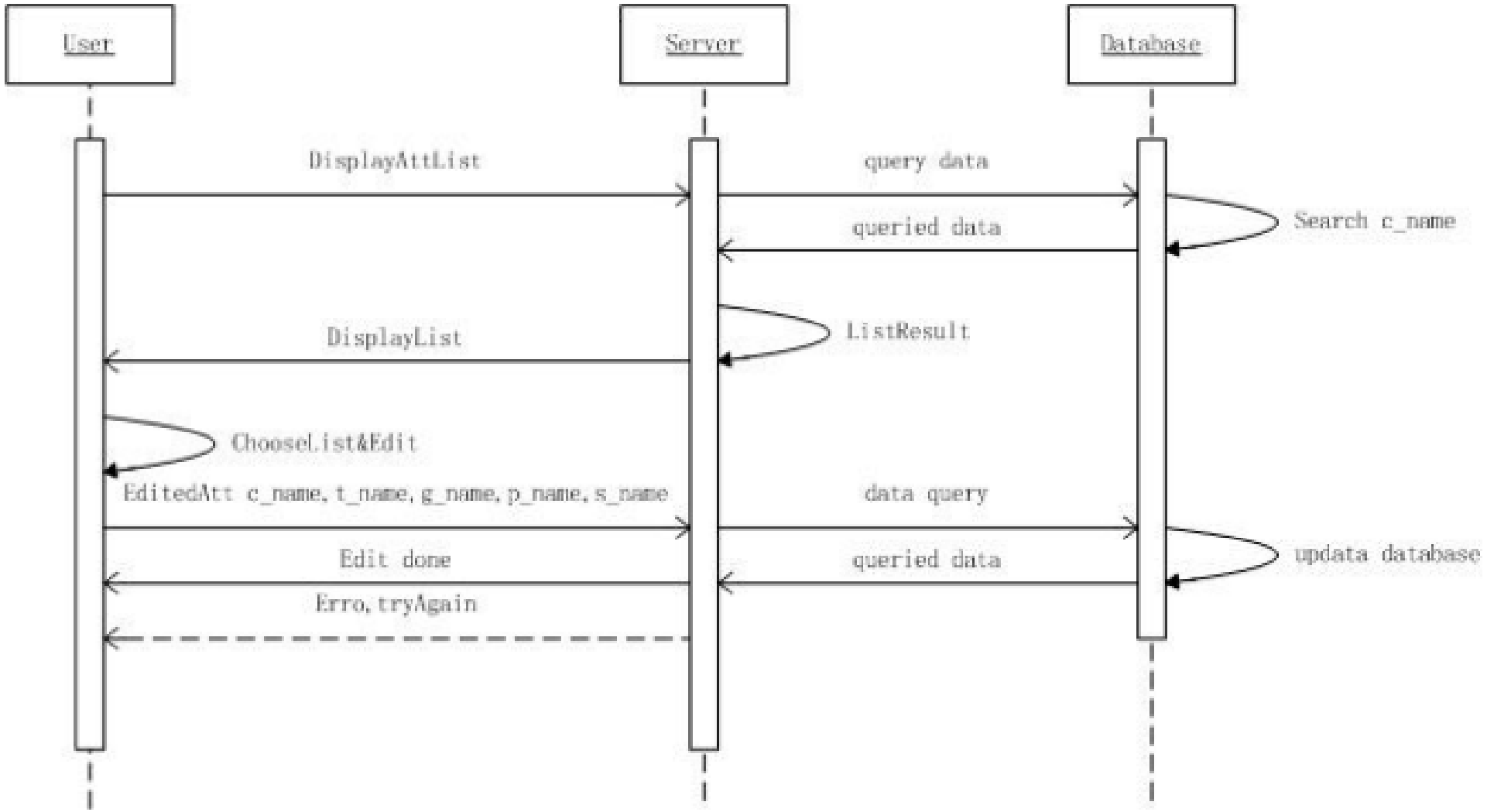
Scenarios for AMS

- **Scenario 2** : Update Student or faculty profile.



Scenarios for AMS

- **Scenario 3** : Update Attendance



Guidelines for Sequence Models

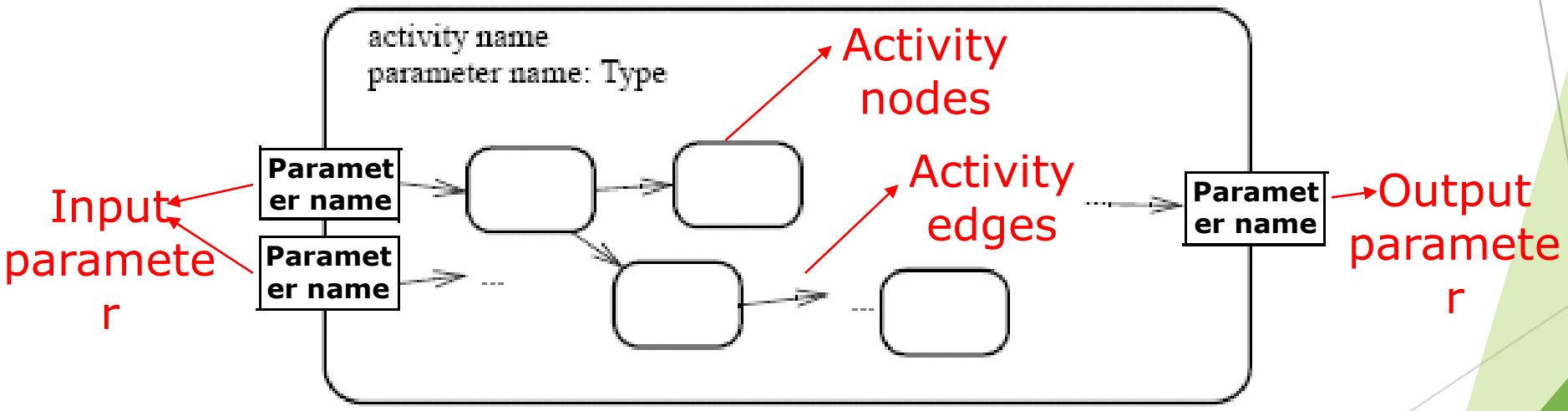
- Prepare at least one scenario per use case:
- Abstract the scenarios into sequence diagrams: The seq. diagram clearly show the contribution of each actor.
- Divide complex interaction:
- Prepare a sequence diagram for each error condition.

Activity Diagrams

- Useful to specify software or hardware system behaviour
- Based on data flow models – a graphical representation (with a Directed Graph) of how data move around an information system

Activities

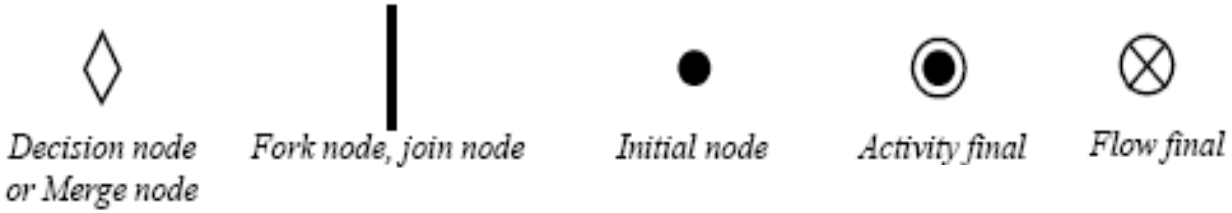
- An activity is the specification of parameterized behaviour as the coordinated sequencing of subordinate units whose individual elements are actions
- Uses parameters to receive and provide data to the invoker



Activity nodes

- Three type of activity nodes:
  - **Action nodes**: executable activity nodes; the execution of an action represents some transformations or processes in the modeled system (already seen)

  - **Control nodes**: coordinate flows in an activity diagram between other nodes



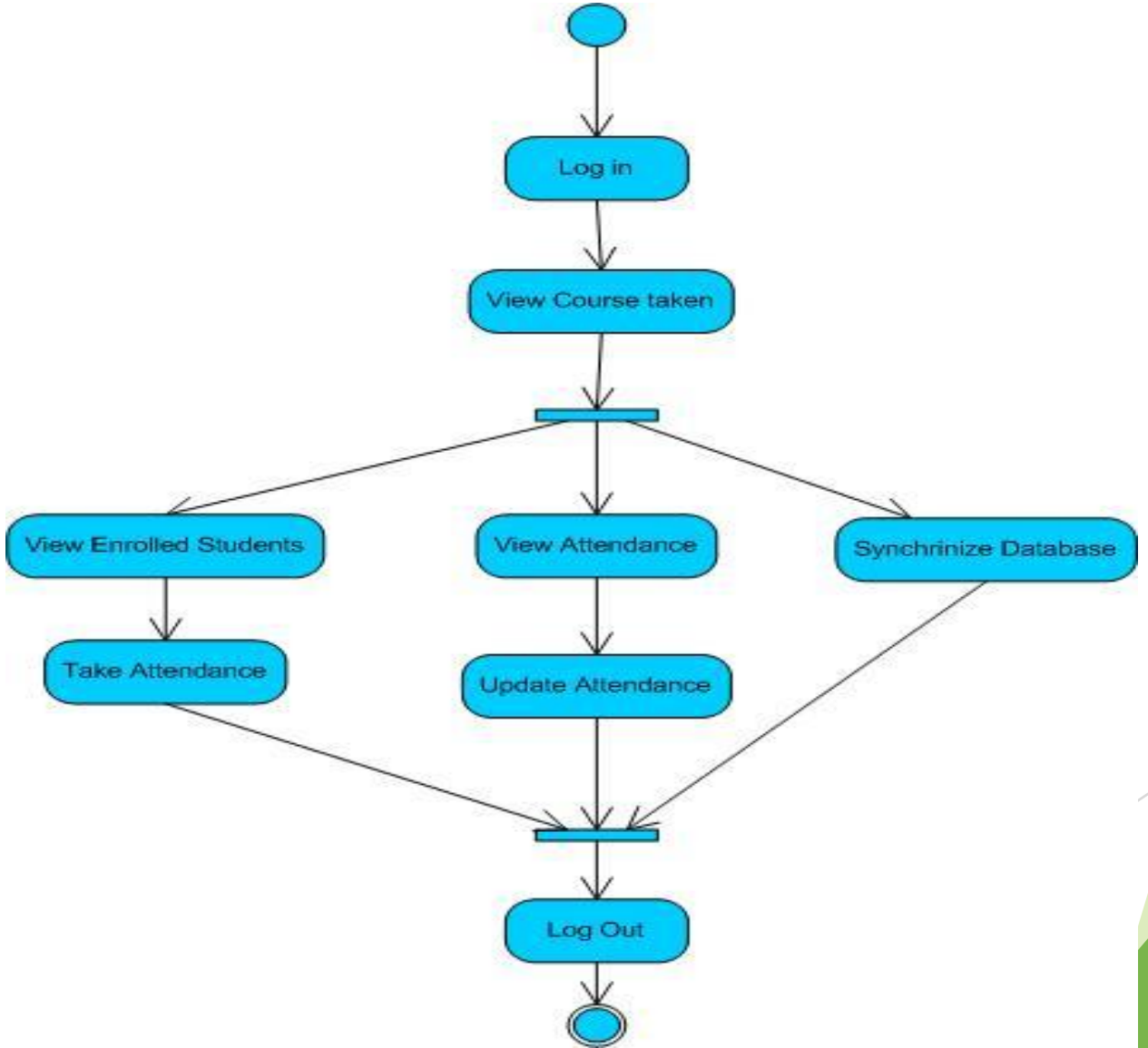
Forking and Joining

- Use a synchronization bar to specify the forking and joining of parallel flows of control
- A synchronization bar is rendered as a thick horizontal or vertical line.

Fork-A fork may have one incoming transitions and two or more outgoing transitions

- each transition represents an independent flow of control
- conceptually, the activities of each of outgoing transitions are concurrent either truly concurrent (multiple nodes) or sequential yet interleaved (one node)

Forking and Joining(Cont'd)

- **A join** may have two or more incoming transitions and one outgoing transition above the join, the activities associated with each of these paths continues in parallel at the join, the concurrent flows synchronize
  - Each waits until all incoming flows have reached the join, at which point one flow of control continues on below the join

Activity Diagram of Attendance Management System